arXiv:2604.24468v1 [cs.CR] 27 Apr 2026

A Survey on Split Learning for LLM Fine-Tuning: Models, Systems, and Privacy Optimizations

ZIHAN LIU, Zhejiang University, China
YIZHEN WANG, Zhejiang University, China
RUI WANG, Zhejiang University, China
XIU TANG, Zhejiang University, China
SAI WU, Zhejiang University, China

Fine-tuning unlocks large language models (LLMs) for specialized applications, but its high computational cost often puts it out of reach for resource-constrained organizations. While cloud platforms could provide the needed resources, data privacy concerns make sharing sensitive information with third parties risky. A promising solution is split learning for LLM fine-tuning, which divides the model between clients and a server, allowing collaborative and secure training through exchanged intermediate data, thus enabling resource-constrained participants to adapt LLMs safely. In light of this, a growing body of literature has emerged to advance this paradigm, introducing varied model methods, system optimizations, and privacy defense-attack techniques for split learning. To bring clarity and direction to the field, a comprehensive survey is needed to classify, compare, and critique these diverse approaches. This paper fills the gap by presenting the first extensive survey dedicated to split learning for LLM fine-tuning. We propose a unified, fine-grained training pipeline to pinpoint key operational components and conduct a systematic review of state-of-the-art work across three core dimensions: model-level optimization, system-level efficiency, and privacy preservation. Through this structured taxonomy, we establish a foundation for advancing scalable, robust, and secure collaborative LLM adaptation.

CCS Concepts: • **General and reference → Surveys and overviews**; • **Computing methodologies → Distributed computing methodologies**; **Natural language processing**; • **Security and privacy → Privacy protections**.

Additional Key Words and Phrases: Large Language Models Fine-Tuning, Split Learning, Split Federated Learning, Collaborative Training, Privacy-Preserving Mechanisms, System Optimization

ACM Reference Format:
Zihan Liu, Yizhen Wang, Rui Wang, Xiu Tang, and Sai Wu. 2018. A Survey on Split Learning for LLM Fine-Tuning: Models, Systems, and Privacy Optimizations. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 35 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Large Language Models (LLMs) such as GPT [8, 115], Qwen [154], and DeepSeek [84] have demonstrated exceptional capabilities in general-purpose tasks. To adapt these models to specific domains, fine-tuning has emerged as the mainstream paradigm, effectively transitioning them from *generalists* to specialized *domain experts*. However, with

Authors' Contact Information: Zihan Liu, Zhejiang University, Ningbo, China, zihanliu24@zju.edu.cn; Yizhen Wang, Zhejiang University, Ningbo, China, wyzwalker@zju.edu.cn; Rui Wang (corresponding author), Zhejiang University, Ningbo, China, rwang21@zju.edu.cn; Xiu Tang, Zhejiang University, Ningbo, China, tangxiu@zju.edu.cn; Sai Wu, Zhejiang University, Ningbo, China, wusai@zju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to ACM

the increasing scale of model parameters and the gradual depletion of high-quality public datasets, fine-tuning on private data faces the dual challenges of severe resource constraints and privacy leakage risks. Modern LLMs typically comprise tens to hundreds of billions of parameters [100], rendering full fine-tuning prohibitively expensive for small and medium-sized enterprises (SMEs) with limited computational budgets. Although Parameter-Efficient Fine-Tuning (PEFT) techniques mitigate these costs by updating only a small subset of parameters, the substantial GPU memory requirements for loading and serving large models remain a significant barrier. Furthermore, fine-tuning on proprietary data raises privacy concerns regarding data outsourcing: enterprises are often hesitant to upload sensitive data to third-party cloud servers due to risks of direct exposure and potential breaches.

To address these key bottlenecks, split learning (SL) [42, 134] has emerged as a promising distributed learning paradigm. In SL, the model is partitioned between clients and a central server: clients retain and execute only a small portion of the model (typically the front-end and/or back-end model layers), while the computationally intensive middle layers are offloaded to the server. Training proceeds via the exchange of intermediate representations (smashed data) and gradients rather than raw data, thereby significantly reducing local computational and storage burdens while preserving data privacy. Recently, combining SL with LLM fine-tuning has attracted growing research attention [12, 76, 81, 91]. By applying this split-based paradigm to large language models, resource-constrained clients can participate in fine-tuning billion-parameter models while maintaining only a small fraction of parameters locally. This approach enables small and medium-sized enterprises and research institutions to collaboratively train or fine-tune private large models without exposing sensitive data to external parties, effectively addressing both resource and privacy challenges simultaneously.

Integrating split learning with LLM fine-tuning (split-based LLM-FT) has enormous potential and has spurred a wealth of research across three complementary directions. First, a substantial line of work targets model-level optimization challenges in multi-client federated scenarios, addressing convergence degradation caused by biased intermediate features due to data heterogeneity (Non-IID), server-client update imbalance arising from increased server-side batch sizes when aggregating smashed data, and catastrophic forgetting during sequential training or partial client participation. Second, another research stream focuses on system-level efficiency improvements, tackling severe communication bottlenecks from frequent transmission of massive intermediate activations and gradients, resource utilization imbalances caused by server-client coordination overhead, and the *straggler effect* induced by heterogeneous client computing power and network bandwidth. Third, a growing body of literature investigates privacy and security concerns, encompassing both attack methodologies that exploit semantically rich gradients and smashed data to reconstruct private information, as well as the distributed framework's exposed model manipulation interfaces to inject malicious behaviors, and their corresponding defense strategies.

Several surveys on split learning have already been published, as summarized in Table 1. For instance, [52] categorizes SL paradigms into model-only segmentation, weight-based aggregation, and intermediate data aggregation methods, evaluating their performance under IID and non-IID scenarios. [78] surveys resource-efficient frameworks and optimization strategies for alleviating computational and communication bottlenecks caused by stragglers. [62, 120] summarize attack and defense approaches in split learning, while [41] evaluates perturbation-based and learning-based defenses against data reconstruction and label inference attacks and their impact on downstream performance.

However, despite these foundational efforts, existing surveys suffer from three critical gaps in the context of LLM adaptation. First, current surveys primarily focus on traditional, small-scale models, overlooking the distinct challenges of integrating split learning with LLMs. From a model perspective, they assess optimization strategies in conventional tasks without verifying their efficacy for LLM fine-tuning. At the system level, they neglect the unique architectural demands, massive parameter scales, and performance constraints of LLMs. Regarding privacy, they overlook the

Table 1. Comparison of related surveys and research works of split learning.

Paper	Year	Focus Areas			Unified Training Pipeline	Component-Level Bottleneck	LLM
		Model Performance	System Optimization	Privacy			
[24]	2022	✓	✓	✓			
[78]	2024		✓				
[53]	2025	✓	✓				
[117]	2025	✓	✓				
[52]	2025	✓	✓				
[56]	2025	✓	✓	✓			
[120]	2025			✓			
[41]	2025			✓			✓
[62]	2025			✓			
Ours	-	✓	✓	✓	✓	✓	✓

exacerbated vulnerabilities of auto-regressive LLMs, where training labels act as shifted input sequences, making label leakage mathematically equivalent to exposing raw proprietary text. Second, previous surveys treat split learning paradigms as macro-level architectures without establishing a unified framework for split-based LLM fine-tuning. They lack an explicit, component-level decomposition of the computational and communication steps involved in split-based LLM fine-tuning, leaving the training pipeline ambiguous. Third, current surveys do not systematically map multidimensional challenges and existing solutions to the specific execution stages of LLM adaptation. Without a unified training pipeline, there is no structured way to anchor inefficiencies, model degradations, privacy threats, and their corresponding countermeasures to precise operational components. This omission hinders the development of a comprehensive taxonomy needed to guide component-level design and future research directions.

To bridge this critical gap, this paper presents the first comprehensive review focused exclusively on the intersection of split learning and LLM fine-tuning. Our primary contributions are threefold:

- A unified split-based LLM-FT pipeline: Diverging from previous coarse classifications, we abstract and decompose existing methods into a unified, highly granular training pipeline. This structural breakdown enables precise analysis of component-level bottlenecks and future optimization trajectories.
- multi-dimensional bottleneck analysis: We map critical vulnerabilities directly to specific execution steps, systematically exposing the root causes of convergence degradation, system inefficiencies, and privacy threats in split-based LLM-FT.
- component-anchored literature taxonomy: By mapping multidimensional challenges and state-of-the-art optimization methods directly onto our unified pipeline, we deliver a comprehensive taxonomy. This framework explicitly anchors system inefficiencies, model degradations, and privacy countermeasures to specific execution steps, systematically guiding future research.

The rest of the article is structured as follows and the structure is in Figure 1. Section 2 provides a comprehensive overview of split-based LLM-FT, detailing model partitioning strategies, overarching frameworks, and our proposed unified pipeline components. Section 3 systematically analyzes strategies for model-level optimization and performance enhancement. Section 4 explores current methodologies for system-level optimization. Section 5 delves into the privacy and security landscape, categorizing prominent attack vectors and their corresponding defense mechanisms within split-based architectures. Finally, Section 6 concludes this survey.

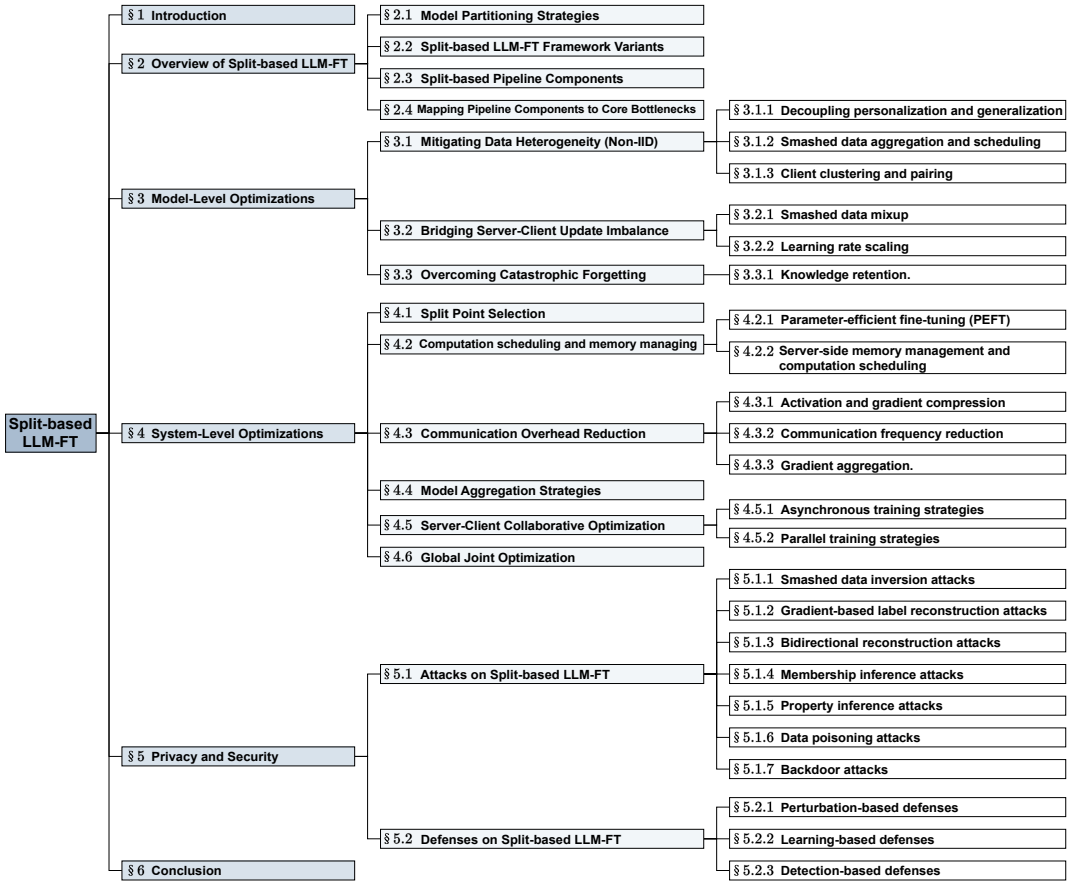


Fig. 1. Survey framework: from a unified training pipeline to a multidimensional taxonomy of system, model, and privacy.

2 Overview of Split-based LLM-FT: Model Partition, Framework Variants, and Pipeline Components

2.1 Model Partitioning Strategies

Figure 2 illustrates two primary model partitioning paradigms: Vanilla SL and U-shaped SL.

Two-part split (Vanilla SL) [42, 59, 134] partitions the model into two parts (head model-trunk model), deploying the majority of model layers on a remote server while retaining a small portion locally. During each forward pass, the client transmits forward intermediate activations and labels to the server. The server then computes the loss and performs backpropagation, returning gradients to the client. The client then performs local backpropagation. This strategy focuses on reducing computational and storage burdens on the client.

Three-part split (U-shaped SL) [11, 45, 95] divides the model into three parts (Head Model-Trunk Model-Tail Model), keeping loss calculation and labels locally. This architecture inherently enhances privacy by preventing direct label exposure. This protection is particularly critical in LLM supervised fine-tuning (SFT); because SFT employs teacher forcing, where labels are merely left-shifted input sequences that encode the complete raw text, localizing the loss computation is imperative to thwart server-side data reconstruction. However, compared to Vanilla SL, U-shaped SL strictly doubles the communication volume within a single forward-backward pass, as intermediate activations and gradients must traverse the network twice.

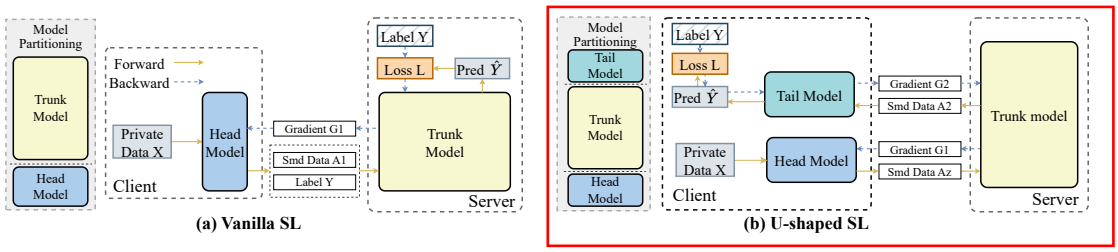


Fig. 2. Two distinct model partitioning strategies.

2.2 Split-based LLM-FT Framework Variants

Based on different training strategies, the split learning paradigm can be categorized into several training frameworks, as illustrated in Figure 3. Although these architectural variants were initially designed for conventional split learning tasks, their coordination mechanisms can be seamlessly adapted to collaborative LLM fine-tuning scenarios.

Sequential split learning (Sequential SL) [42, 134]. Sequential SL is the earliest proposed Split Learning framework, in which clients train sequentially with the server. After completing their local training, each client transfers its updated local model to the next client in a predefined order. In this manner, clients take turns training on their respective data partitions, forming a serialized training workflow.

Parallel split learning (PSL) [57, 83]. To address the inefficiency of Sequential SL, where most clients remain idle while waiting for their turn, PSL enables clients to train in parallel. The server aggregates the smashed data uploaded by all clients to update the server-side model and simultaneously returns either personalized or globally aggregated gradients to each client. Clients then update their local models using the received gradients. This strategy effectively improves computational efficiency and reduces idle time among clients.

Split federated learning (SFL) [37, 59]. In SFL, all clients train their local parts of the model in parallel and update them based on their computed gradients. After a certain number of training rounds, each client sends its local model to a central federated server, which aggregates the models to form a global client model and then redistributes it. Unlike PSL, SFL introduces periodic global aggregation of client models, providing stronger global consistency but with additional synchronization overhead. Specifically, SFL encompasses two server-side update variants: SFL-V1 updates the global trunk model by aggregating gradients from all clients via FedAvg, whereas SFL-V2 processes client activations sequentially in random order, executing immediate server model updates without global aggregation.

Group-based split federated learning (GSFL) [73, 89, 144, 171]. To address the efficiency bottlenecks and data heterogeneity inherent in standard SFL, GSFL restructures the flat client-server topology into a clustered architecture. From a system perspective, clients with similar computational and network capabilities are grouped to mitigate the straggler effect and reduce synchronization delays. From a data perspective, clients with similar data distributions are clustered to counteract non-IID challenges, thereby alleviating client drift and preventing model divergence.

Other framework variants. Based on the concept of split learning, several other novel frameworks have been proposed. RingSFL [122] achieves load balancing and effectively mitigates the bottleneck effect by constructing clients into a ring topology and adaptively distributing model layers based on each client's computational capabilities. HSFL [29, 80, 169] employs a hierarchical multi-server architecture (cloud–edge–device) to overcome the memory and communication bottlenecks of conventional single-layer split learning systems under large-scale user participation. PairingFL [158] lets paired clients train collaboratively, performing most updates within pairs to reduce server communication overhead. CLICOOPER [21] introduces a serverless, multi-client SL framework for partially trusted environments, where a data-owning client delegates sequential model training to a chain of compute-contributing nodes.

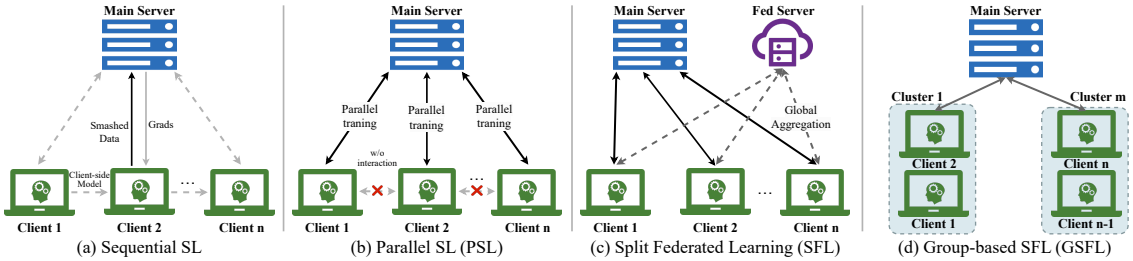


Fig. 3. Split-based LLM-FT framework variants, categorized by training coordination and aggregation mechanisms.

2.3 Split-based Pipeline Components

In the split-based LLM-FT framework, model training relies on the coordinated operation of multiple critical components. Optimization efforts in this domain primarily focus on improving one or more of these components to enhance overall efficiency, performance, and privacy. This section first introduces a unified pipeline, as illustrated in Figure 4, which outlines the complete training workflow. Subsequently, we present a detailed analysis of the key components within this pipeline, providing a foundation for later discussions on component-level optimization strategies. In this unified pipeline, steps 1–4 and 9–10 constitute the core workflow shared by all split-based LLM-FT frameworks. To clearly delineate framework-specific operations, we append a dagger (†) to steps 5–8 to denote procedures exclusive to the three-part split (u-shaped SL) architecture, and a hash symbol (#) to step 11 to indicate the global synchronization mechanism uniquely required by split federated learning (SFL) paradigms.

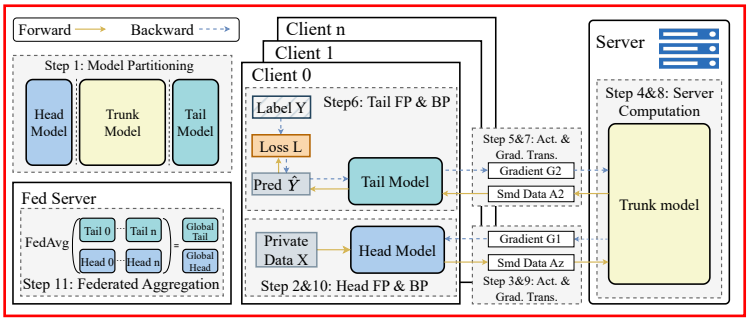


Fig. 4. Unified split-based LLM-FT pipeline.

Step 1: model partition. The complete LLM is typically partitioned at the granularity of embedding layers or individual transformer blocks. Depending on the architecture, it is divided into either two components (head model and trunk model for vanilla SL) or three components (head model, trunk model, and tail model for u-shaped SL). These components are then deployed across the client and server sides according to the adopted split strategy.

Step 2: local head model forward propagation. The client performs forward propagation on the local head model using its local dataset, generating head model's output intermediate activations.

Step 3: intermediate activation transmission (client to server). The client transmits the intermediate activations produced by the head model to the server over the network.

Step 4: server-side processing. Upon receiving the intermediate activations, the server continues the forward propagation through the trunk model. In vanilla SL, the server directly computes the loss using transmitted labels and initiates backpropagation. Conversely, in u-shaped SL, the server suspends the process after the forward pass, awaiting gradients from the client. In both scenarios, the server ultimately computes the gradients corresponding to the intermediate activations output by the head model.

Step 5†: intermediate activation transmission (server to client). Because the server in a u-shaped SL architecture cannot directly access the original labels to compute the loss, it must transmit the intermediate activations produced by the trunk model back to the client for further processing.
Step 6†: local tail model forward and backward propagation. After receiving the intermediate activations from the server, the client performs forward propagation through its local tail model, computes the loss using local labels, and executes backpropagation to obtain the gradients with respect to the trunk model's output activations.
Step 7†: intermediate gradient transmission (client to server). The client transmits the gradients of the trunk model's output activations to the server over the network.
Step 8†: server-side trunk model backward propagation. Upon receiving the gradients from the client, the server continues the backward propagation through the trunk model to compute the gradients corresponding to the head model's output activations.
Step 9: intermediate gradient transmission (server to client). The server sends the gradients of the head model's output activations back to the client.
Step 10: local head model backward propagation. After receiving the gradients of the head model's output activations, the client performs backward propagation on the local head model to update its parameters.
Step 11#: federated aggregation (for SFL-based frameworks). After several local training rounds, each client uploads its local model parameters to the federated server for global aggregation. The federated server aggregates these local models into a global model and redistributes it to all participating clients for the next training round.

2.4 Mapping Pipeline Components to Core Bottlenecks

While the unified pipeline (steps 1–11) establishes a functional paradigm for split-based LLM-FT, mapping its sequential execution onto a distributed, heterogeneous, and potentially untrusted environment exposes critical vulnerabilities during actual deployment. By analyzing each step, we identified bottlenecks across three distinct dimensions, which also dictate the taxonomy of our subsequent sections:

- **Model-level bottlenecks.** In multi-client scenarios, executing local forward and backward passes (steps 2, 6, 10) on highly skewed, Non-IID datasets generates biased smashed data. When aggregated at the server or during global synchronization (step 11), this heterogeneity, coupled with server-client update imbalances, severely degrades model convergence and generalization. Furthermore, in sequential training or scenarios with partial client participation, the model is prone to catastrophic forgetting, inadvertently overwriting previously learned representations while optimizing for the distinct data distributions of current participants. Section 3 explores advanced strategies to rectify these performance degradations.
- **System-level bottlenecks.** At the initiation of the pipeline (step 1), the selection of the split point dictates the local memory footprint and profoundly impacts training latency and energy consumption due to client-server computational disparities. Subsequently, the reliance on continuous, bidirectional transmission of massive intermediate activations (steps 3, 5) and gradients (steps 7, 9) induces severe communication overhead. Furthermore, the central server (step 4) easily becomes a computational bottleneck when coordinating heterogeneous clients. Finally, frameworks that necessitate periodic global model aggregation (step 11) introduce rigid synchronization barriers. Under system heterogeneity, this leads to the straggler problem, where high-capacity clients experience significant training delays waiting for the slowest devices. Section 4 systematically reviews system-level optimizations designed to alleviate these structural inefficiencies.

- **Privacy and security vulnerabilities.** Although the pipeline inherently avoids the direct exchange of raw inputs and labels, the transmitted intermediate representations remain highly vulnerable. Specifically, a malicious server can exploit the intercepted smashed data (steps 3, 5) to launch smashed data inversion attacks, and it can utilize the backpropagated gradients (steps 7, 9) to execute gradient-based label reconstruction. Furthermore, because the server maintains control over a substantial segment of the model architecture and training process (step 4), it can actively manipulate the training process to initiate data poisoning and backdoor attacks. To counter these vulnerabilities, Section 5 details the prevailing threat landscape and systematically summarizes the corresponding privacy-preserving defense mechanisms.

3 Model-Level Optimization and Performance Enhancement

Model-level optimizations are pivotal for ensuring the effectiveness of Split-based LLM Fine-tuning, particularly in terms of convergence, accuracy, and robustness. Theoretically, in a standard single-client, single-server architecture, the complete training process of split learning is mathematically equivalent to centralized training, yielding nearly identical model performance. However, when scaled to collaborative fine-tuning scenarios involving multiple clients, this equivalence is fundamentally compromised. Specifically, three primary challenges emerge that undermine model performance:

- **Challenge 1: data heterogeneity (non-IID).** The private data of each client often exhibits divergent distributions. The intermediate features generated from these non-IID datasets can bias the convergence trajectory of the server-side model, degrading global model accuracy.
- **Challenge 2: server-client update imbalance.** The server-side model processes aggregated smashed data from multiple clients, effectively learning from a "larger batch" than any single client. This causes the server to update faster than client-side models, leading to desynchronized convergence.
- **Challenge 3: catastrophic forgetting.** In sequential training or scenarios with partial client participation, the model tends to inadvertently overwrite representations learned from previous clients while optimizing for the distinct data distributions of current participants.

This section reviews recent methodologies designed to address these challenges through personalization, feature distribution alignment, and advanced optimization strategies. A comprehensive summary of these model-level optimization strategies is presented in Table 2. Although several foundational methods lack empirical validation on native LLM tasks, we have theoretically evaluated their algorithmic compatibility. Based on their core mechanisms, we project their suitable LLM applications and categorize them into Natural Language Understanding (NLU) and Generation (NLG), as detailed in the *Applicabiliy* column.

3.1 Mitigating Data Heterogeneity (Non-IID)

The inherent statistical divergence among client datasets poses a severe threat to the convergence and generalization of the global model. To address the feature shifts induced by non-independent and identically distributed (Non-IID) data, recent literature predominantly explores three main strategies: decoupling personalization and generalization via architectural design, smashed data aggregation and scheduling, and client clustering and pairing.

3.1.1 Decoupling personalization and generalization via architectural design. In scenarios where client data exhibits non-IID characteristics, client-side models are updated exclusively on skewed local datasets. Consequently, the generated

Table 2. Summary of model-level optimization and performance enhancement methods in split-based Learning. LLM Exp. indicates whether the original paper empirically evaluated the method on LLM datasets (✓). Applicability represents our theoretical assessment of the method's viable scope in the LLM era, categorized into Natural Language Understanding (NLU) and Generation (NLG) tasks.

Target Challenge	Approach	Method	Year	LLM Exp.	Applicability	Key Innovation / Contribution
Challenge 1: Data Heterogeneity	Personalization & Generalization	SplitGP [44]	2023	-	NLU+G	Dual-task loss for personalization/generalization; λ-based aggregation.
		PFSL [136]	2023	-	NLU+G	Two-stage training with Work Fairness Constraint for load balancing.
		DualFed [180]	2024	-	NLU+G	Dual-feature representation with separate encoders for global/local tasks.
		FSMKD [93]	2024	-	NLU+G	Two-body structure with Mutual Knowledge Distillation (soft labels).
		ESL [11]	2024	-	NLU+G	Two-stage training targeting non-IID data optimization.
		SplitLPF [13]	2024	-	NLU+G	Contribution-based fair aggregation for U-shaped structures.
		FlexP-SFL [163]	2025	✓	NLU+G	KL divergence regularization to align intermediate representations.
	Smashed Data Aggregation and Scheduling	MiniBatch-SFL [54]	2023	-	NLU+G	Centralized gradient averaging of smashed data to remove client drift.
		MergeSFL [72]	2024	-	NLU	Feature merging for non-IID; Joint optimization with genetic algorithms.
		SCALA [156]	2024	-	NLU	Concatenated activations with Logit Adjustment for long-tail data.
		S2FL [153]	2024	-	NLU	Re-grouping server-side features by label to simulate IID batches.
		Hourglass [46]	2025	-	NLU+G	LSH-based "dissimilar priority" scheduling for asynchronous processing.
		Ampere [173]	2025	-	NLU+G	Unified activation merging to reduce heterogeneity.
	Client Clustering and Pairing	CSFL [151]	2024	-	NLU	Clustering clients based on label distribution similarity and adaptively applying global or personalized aggregation.
		ParallelSFL [73]	2024	✓	NLU	Clustering clients to ensure the assembled data distribution within groups closely approximates the global IID.
		PairingFL [158]	2025	-	NLU	Selecting clients with dissimilar and diverse data distributions to maximize marginal gain for each training round.
		GSFL [89]	2025	-	NLU+G	Grouping devices with similar data distributions measured via cosine similarity on encrypted intermediate features.
Challenge 2: Update Imbalance	Learning Rate Scaling	SGLR [106]	2022	-	NLU+G	Server learning rate acceleration based on client count power scaling.
		musfl [69]	2022	-	NLU+G	Coordinated Local-Global Learning Rate Compensation.
	Smashed Data Mixup	LocFedMix-SL [104]	2022	-	NLU	Smashed data mixing and mutual information regularization.
		Mix2SFL [103]	2024	-	NLU	Two-way manifold mixup on paired smashed data.
		GAS [155]	2025	-	NLU	Generative Gaussian distribution for virtual activation sampling.
		SMixSL [130]	2025	-	NLU	Cutout, CutMix, and Mixup applied to smashed features.
Challenge 3: Catastrophic Forgetting	Knowledge Retention	MultiSFL [148]	2025	-	NLU+G	Helper clients extract and replay missing knowledge.
		SLwF [30]	2025	-	NLU+G	Review/Distinguish/Decelerate modules for continual learning.

intermediate features often lack representativeness for global classes (or fail to capture meaningful information for out-of-distribution categories). This feature shift not only impedes the effective generalization of the server-side model but also renders a simple globally aggregated model insufficient for adapting to the personalized distributions of individual clients. Consequently, recent methods [11, 13, 44, 93, 136, 163, 180] focus on reconciling the conflict between personalization and generalization in SFL scenarios. SplitGP [44] decouples the model into a client-side component dedicated to strong personalization and a server-side component focused on strong generalization. By jointly optimizing both components via a multi-exit objective function and a hybrid aggregation strategy, the framework achieves robustness to both local and out-of-distribution data while significantly reducing inference latency. Similarly, DualFed [180] addresses this dilemma by exploiting the hierarchical nature of deep representations to decouple generalization from personalization. Instead of structurally splitting the model across devices, it inserts a personalized projection network between the shared encoder and the classifier. This architecture yields dual representations: pre-projection features, which retain generalized information to support a global classifier, and post-projection features, which are mapped to a personalized space for local adaptation. To prevent the optimization of local tasks from compromising the generalized features, it employs a stage-wise training strategy that alternates between updating the personalized main branch and the global classifier. SFML [147] maintains a dedicated privacy model and the split segments (head and tail) of a public model locally on each client, and leverages a mutual learning module to enable bidirectional knowledge transfer between the two models, perfectly balancing lightweight computation, personalization, and global generalization.

3.1.2 Smashed data aggregation and scheduling. Beyond balancing personalization and generalization, another effective paradigm for mitigating client-side data heterogeneity is the direct regulation of intermediate activations. By merging or scheduling the smashed data uploaded by different clients, the server can smooth out the statistical variance between

batches. This ensures that the server-side model observes a more representative and balanced global distribution during training, preventing it from overfitting to any single client's skewed domain.

Smashed data aggregation. Some methods [54, 72, 153, 156, 173] use Feature Merging to make the server-side feature batches approximate the IID distribution of feature batches. MergeSFL [72] addresses statistical heterogeneity by constructing a mixed feature sequence that approximates an IID distribution. specifically, the server dynamically selects workers under bandwidth constraints to minimize the KL divergence of the combined data. By merging features from these selected workers, the framework rectifies the top model's update direction, significantly enhancing final accuracy. SCALA [156] simulates centralized training to mitigate local label skew by concatenating client-side activations at the server. Furthermore, it addresses the global long-tail distribution caused by partial participation through logit-adjusted loss functions, thereby effectively improving SFL performance on Non-IID data. However, applying these strategies to native LLMs introduces severe memory bottlenecks. Given the massive size of LLM activations, concatenating smashed data from multiple clients rapidly exhausts server GPU capacity, triggering catastrophic out-of-memory failures.

Smashed data scheduling. Taking a data-content-centric approach, Hourglass [46] introduces a "Dissimilar Priority Scheduling" (DPS) mechanism. It utilizes Locality-Sensitive Hashing (LSH) to efficiently identify intermediate features that differ significantly from those currently being processed. By prioritizing these "dissimilar" features during server-side training, Hourglass maximizes the information gain of each update step, thereby accelerating convergence even under severe data heterogeneity.

3.1.3 Client clustering and pairing. Smashed data aggregation and scheduling alleviate data heterogeneity after the server receives the intermediate features, while client clustering and pairing preemptively reorganize the collaborative structure based on statistical characteristics, thereby making the training data within a group approximate an IID distribution. To eliminate the impact of non-IID data on model accuracy, ParallelSFL [73] partitions participating workers into multiple clusters , requiring the assembled data distribution of each cluster to closely approximate the global IID distribution. It introduces KL-divergence to measure the distribution gap. The algorithm first uses K-means to group similar workers , then greedily selects workers across these groups to build clusters , and finally fine-tunes by exchanging workers across clusters to minimize the KL-divergence of each cluster. PairingFL [158] adopts a finer-grained collaboration strategy. Specifically, while ensuring participation fairness across all clients , it employs a greedy algorithm to iteratively select clients with the maximum marginal gain (i.e., those with the most dissimilar and diverse data distributions) to form the training subset for each round. GSFL [89] uses a lightweight hash encryption function to encrypt the intermediate features of devices and sends them to the server. The server uses Cosine Similarity to measure the similarity of data distributions among different devices, and assigns devices with similar data distributions into the same group, thereby mitigating the negative impact brought by Non-IID data. Similarly, CSFL [151] evaluates the similarity of clients' label distributions using cosine similarity, and subsequently employs the K-means algorithm to partition them into K clusters. During the training phase, the framework adaptively toggles between a global aggregation strategy and a personalized cluster-specific approach depending on the degree of inter-cluster heterogeneity.

3.2 Bridging Server-Client Update Imbalance

In multi-client scenarios, the server effectively processes a dynamically expanding batch aggregated from concurrent participants, leading to a profound desynchronization between server and client learning trajectories. To harmonize this update imbalance, optimization efforts have focused on feature-level augmentation and learning rate scaling.

3.2.1 Smashed data mixup. In multi-client parallel split learning architectures (e.g., SFL, PSL), the server-side model processes aggregated smashed data from n concurrent clients in each communication round. Consequently, the server-side model experiences an effective learning rate that is amplified by a factor of n relative to the client-side models. This server-client Update Imbalance induces training inconsistencies, as a higher effective learning rate typically necessitates a proportionally larger batch size to preserve the stability of gradient estimation. To mitigate this discrepancy, several approaches [103, 104, 130, 155] employ data augmentation directly on smashed data to harmonize the learning speeds of the clients and the server. LocFedMix-SL [104] proposes applying the Mixup technique at the level of intermediate activations. By synthesizing augmented smashed data via linear interpolation, this method effectively expands the server's virtual batch size to align with its amplified learning rate, thereby stabilizing the system's training dynamics. Building on this, Oh et al. [103] proposed Mix2SFL, a framework that synchronizes forward and backward flows via a "two-way mixup" strategy. Specifically, it employs SmashMix to combinatorially mix smashed data, balancing the server-client update ratio with averaged forward flows. Furthermore, Mix2SFL incorporates GradMix to average split-layer gradients, which not only aligns backward signals but also enhances communication efficiency by enabling downlink broadcasting. While potentially viable for natural language understanding (NLU) tasks, this strategy is fundamentally incompatible with natural language generation (NLG) tasks, as interpolating variable-length hidden states and shifted next-token labels destroys semantic integrity and triggers convergence collapse.

3.2.2 Learning rate scaling. Due to the server-side large effective batch problem, where the effective batch size at the server is the sum of all client batches resulting in an update imbalance between the server and the clients, SGLR [106] splits the learning rate into server-side and client-side components and independently accelerates the server-side learning rate. MU-SplitFed [69] binds the server learning rate to the unbalanced step count τ and reduces it to suppress the variance and client drift caused by stale information , while simultaneously amplifying the global aggregation learning rate to compensate for the convergence slowdown caused by the shortened local steps.

3.3 Overcoming Catastrophic Forgetting

In sequential training or scenarios with partial client participation, the model is highly susceptible to overwriting previously acquired representations when adapting to newly introduced skewed distributions.

3.3.1 Knowledge retention. In vanilla SL, the model is trained sequentially across multiple clients with Non-IID local data, which often causes it to adapt to the current device while losing consistency with previously learned knowledge, resulting in degraded performance. To mitigate this issue, SLwF [30] incorporates three lightweight modules: a review module that revisits prior knowledge through contrastive learning, a dividing module that facilitates learning of new or hard classes, and a decelerated update module that stabilizes model updates via an EMA-based strategy. Together, these modules enable SLwF to better balance knowledge retention and adaptation during client switching. In the SFL scenario, if only a portion of the clients participate in each training round, the system may also be prone to catastrophic forgetting and will also be challenged by gradient divergence caused by non-independent and non-IID data. To address these issues, MultiSFL [148] introduces a multi-model aggregation architecture that maintains multiple branch models to promote knowledge sharing. It further mitigates forgetting through a score-vector-based knowledge replay mechanism that leverages idle clients to compensate for missing-category features. Moreover, MultiSFL employs the Federated Gradient Norm (FGN) to identify critical learning periods and adaptively adjust the sampling proportion, thereby balancing model performance and communication overhead.

4 System-Level Optimizations

System-level optimizations are critical for improving the overall efficiency, scalability, and stability of Split-based LLM Fine-Tuning (LLM-FT) frameworks. While model partitioning effectively reduces local computational burdens, it introduces new system challenges such as high communication overhead, unbalanced resource utilization, and synchronization delays. This section provides a comprehensive review of recent efforts aimed at optimizing the system architecture and execution efficiency of Split-based LLM-FT. To provide a holistic view, Fig. 5 illustrates the integration of these strategies within the unified training pipeline introduced in Subsection 2.3. Guided by the execution flow of this pipeline, the following subsections sequentially detail split point selection, computation optimization and resource management, communication overhead reduction, and model aggregation strategies. Finally, the section concludes by exploring server–client collaborative optimization and global joint optimization in the last two subsections. Notably, throughout this section, works denoted with an asterisk (*) are either specifically designed for or experimentally validated on LLMs. Conversely, unmarked methods originate from conventional split learning tasks, yet their underlying principles exhibit strong transferability to LLM fine-tuning scenarios.

4.1 Split Point Selection - Step 1

The split point determines how many layers are computed locally, which must fit within the client's maximum GPU memory. Due to differences in computing power between client and server, the chosen split also affects training latency and energy consumption. Recent studies have explored strategies to optimize split-point selection—for example, by minimizing the overall training delay or reducing energy consumption during distributed training [99, 119, 133]. Additionally, in scenarios involving multiple concurrent clients (such as SFL), each client can select different split points based on its own computational capacity [18]. This approach adapts to individual computational capabilities while balancing training times across clients, thereby mitigating the low training efficiency caused by stragglers [96, 142, 153]. Importantly, a critical distinction exists in native LLMs: unlike CNNs, Transformer blocks maintain a strictly constant hidden dimension. Consequently, shifting the split point does not alter the communication payload size. Therefore, dedicated communication reduction techniques are required to minimize this overhead.

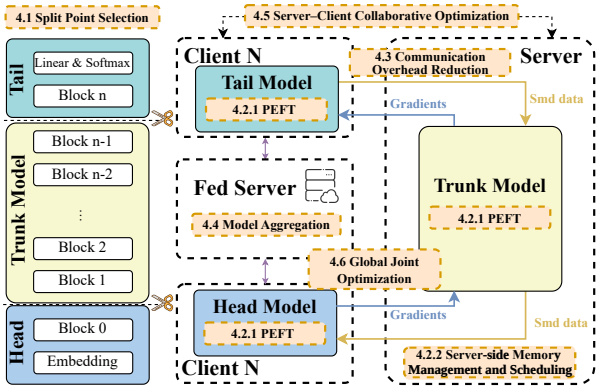


Fig. 5. Overview of system-level optimization for Split-based LLM-FT

4.2 Computation scheduling and memory managing - Step 2 & 4 & 6† & 8†

While model partitioning fundamentally mitigates the burden on individual clients, the sheer scale of LLMs continues to impose severe computational and storage overheads across the entire collaborative system. Consequently, optimizing computation scheduling and memory management requires a comprehensive, two-pronged approach. First, to further minimize the overall memory footprint and computational costs, recent research extensively integrates parameter-efficient architectures (PEFT) across both client and server models. Second, as the pivotal hub executing the Trunk Model and managing concurrent multi-client requests, the central server requires dedicated resource orchestration.

Without efficient scheduling and memory management, the server is highly susceptible to critical system bottlenecks, primarily driven by concurrent GPU memory contention and the *straggler effect*.

4.2.1 Parameter-efficient fine-tuning (PEFT). The direct application of traditional full fine-tuning (FFT) in split learning presents significant challenges due to severe memory and communication bottlenecks. On the one hand, FFT necessitates maintaining full-parameter optimizer states and gradients, imposing prohibitive memory overheads on both resource-constrained clients and the central server. On the other hand, in collaborative training scenarios, the requirement to aggregate the complete model parameters, rather than lightweight updates, significantly exacerbates communication costs. To mitigate these limitations, recent research has extensively integrated parameter-efficient fine-tuning (PEFT) techniques, including low-rank adaptation (LoRA) [55, 76, 81, 96, 176], adapters [6, 51], and prompt tuning [6, 9, 40], into the split learning paradigm. SplitLoRA* [76] is the first SFL framework designed for LLM fine-tuning. By integrating LoRA with aplit federated learning (SFL), it aims to address the computation and communication bottlenecks faced by traditional federated learning on resource-constrained edge devices, utilizing periodic client adapter aggregation to maintain the benefits of parallel training. To accommodate heterogeneous computing environments, HSplitLoRA* [81] proposes a dynamic framework that adapts model split points and LoRA ranks to individual client budgets. It further enhances efficiency through an important weight identification scheme and ensures robust model updates via a noise-free adapter aggregation mechanism using matrix concatenation. Furthermore, Menos* [51] targets the scalability bottleneck on the server side. To mitigate server GPU memory overloads, Menos exploits the frozen nature of base model parameters in adapter-based fine-tuning. It implements a spatial sharing mechanism where a single copy of the base model is shared across multiple clients, thereby eliminating the redundant memory consumption of duplicating massive model parameters for concurrent tasks.

4.2.2 Server-side memory management and computation scheduling. Unlike clients, which handle only local forward and backward propagation, the central server must process massive intermediate data and coordinate multiple concurrent clients. Consequently, the server frequently emerges as the system's computational and memory bottleneck. To mitigate this, recent studies focus on server-side optimizations, specifically memory sharing and priority-based scheduling.

Server memory management and sharing. Fine-tuning LLMs places a tremendous burden on server GPU memory, especially when maintaining unique states (e.g., optimizer states, gradients) for multiple concurrent clients. Menos* [51] mitigates this via a dynamic memory sharing mechanism. By swapping client-specific contexts (e.g., LoRA adapters) to CPU memory during idle periods while sharing a single frozen base LLM on the GPU, it significantly reduces memory footprint and prevents OOM errors under high concurrency. Similarly, Hourglass [46] optimizes resource utilization by decoupling server-side model partitions from the client count. Instead of assigning a unique partition per client, it employs M partitions matching the available GPUs to enable efficient data parallelism. This approach processes client features in groups, eliminating the overhead of frequent partition swapping and accelerating model convergence.

Server priority-based scheduling. In split-based learning frameworks, the server acts as a centralized hub, continuously receiving forward and backward propagation tasks from multiple clients. Due to the inherent heterogeneity in clients' computing power and network bandwidth, the arrival times of these tasks are highly irregular. Consequently, a standard First-In-First-Out (FIFO) scheduling strategy often leads to suboptimal execution orders, causing significant server idle time and pipeline bubbles. To address this, priority-based scheduling aims to reorder the execution of tasks to maximize pipeline parallelism. PipeSFL* [37] introduces a mechanism based on clients' overall lag, scheduling stragglers earlier to hide communication latency. Similarly, Chen et al.* [18] employ a greedy algorithm that prioritizes tasks with longer backward propagation times to optimize server-side sequential training. Beyond heuristics, Tirana et al. [131]

formulate workflow orchestration as a joint client-helper assignment and scheduling problem, proving it is NP-hard and solving it via an ADMM-based decomposition to minimize the training makespan.

4.3 Communication Overhead Reduction - Step 3 & 5† & 7† & 9

Due to the frequent transmission of activation values and gradients between the client and server, communication overhead accounts for a significant portion of the time spent in SL training. In the context of LLMs, this bottleneck is severely exacerbated by the massive dimensionality of their hidden states. To mitigate this excessive communication overhead, recent optimization efforts generally fall into three strategies: (1) activation and gradient compression, (2) communication frequency reduction, and (3) gradient aggregation.

4.3.1 Activation and gradient compression. In Split-based LLM-FT, activation and gradient compression has emerged as the most direct adopted approach to reduce communication overhead. By compressing the intermediate activations transmitted from the client to the server and the gradients sent back during backpropagation, these methods effectively decrease the data volume per iteration and shorten transmission latency. However, compression inevitably leads to partial information loss, which can adversely affect model convergence and overall performance if not properly compensated. Therefore, designing efficient compression techniques that strike a balance between communication efficiency and model accuracy has become a major research focus. Existing activation and gradient compression methods can be broadly categorized into three primary streams: **top-k compression**, **dimensionality reduction**, and **quantization**, alongside **other emerging techniques** (e.g., SVD and Token-based methods). While activation and gradient compression methods were initially pioneered for traditional SL frameworks, they provide foundational theoretical blueprints for mitigating the massive communication bottlenecks in Split-based LLM-FT.

Top-k compression transmits only the most significant k elements of the activation or gradient tensors, thereby reducing the transmitted data while preserving key information. RandTopk [177] introduces randomness into Top-k selection, enabling the model to better explore the feature space, avoid local minima, and achieve higher performance and stability under the same compression ratio. MS [178] compensates for lost values in Top-k sparsification with mask encoding, reducing compression errors and gradient bias to improve Split Learning convergence. FedDFC [35] dynamically adjusts the Top-k compression ratio for each client based on their computing capacity and network conditions, optimizing communication efficiency and reducing waiting time.

Dimensionality reduction [3, 121, 126] utilizes a client-side encoder to map intermediate features into low-dimensional representations and a server-side decoder for reconstruction, significantly lowering communication overhead. BottleNet++ [121] incorporates joint source–channel coding, achieving high compression rates by exploiting DNN tolerance to feature distortion. DynSplit [126] employs a multi-path design where a gating network dynamically selects compression routes based on intermediate features, balancing accuracy and communication efficiency. Building upon dimensionality reduction, DEL* [40] incorporates stochastic n-bit quantization to further reduce communication overhead and ensure strict differential privacy , while leveraging server-side soft prompts to compensate for the performance degradation induced by privacy noise.

Quantization [17, 77, 105, 110, 138] minimizes transmission costs by representing intermediate data with reduced bit-width precision (e.g., 8-bit integers) instead of standard floating-point formats. FedLite [138] further reduces communication by compressing activations via product quantization and mitigating the resulting noise with a gradient correction scheme, achieving high compression ratios with minimal accuracy loss. SplitFC [105] leverages feature dispersion for adaptive compression, integrating standard deviation-based feature-wise dropout with a quantization

scheme sensitive to feature variability. SL-ACC [77] employs Adaptive Channel Importance Identification (ACII) to quantify channel contributions via Shannon entropy and utilizes Channel Grouping Compression (CGC) to cluster channels for adaptive quantization bit-width allocation. By applying extreme quantization (binarization) to local client layers, combined with leakage restriction and differential privacy , B-SL [110] drastically reduces edge computation and communication overheads while maintaining high accuracy. However, the efficacy of such extreme quantization in LLM fine-tuning remains an open question.

Other techniques. Beyond the aforementioned categories, several distinct approaches exist. C3-SL [50] employs circular convolution for batch-wise compression, superimposing multiple quasi-orthogonal samples into a single vector to reduce communication overhead while retaining feature details. SVD-SFT [125] applies Singular Value Decomposition (SVD) to decompose the split-layer feed-forward network (FFN) into low-rank matrices, transmitting only compressed activations to significantly reduce communication overhead. StreamSL* [116] focuses on transmitting only unpadded token embeddings and reconstructing the complete sequence using zero padding at the receiving end, thereby achieving lossless embedding compression and improving sequence-level efficiency.

4.3.2 Communication frequency reduction. Complementary to compression techniques that minimize the data volume per iteration, another research stream focuses on reducing the frequency of client-server synchronization. By decreasing the total number of communication rounds, these methods effectively lower the aggregate communication overhead and latency. Specifically, these approaches primarily encompass three strategies: **intermittent local model update** [3, 17, 101], **local loss updates** [9, 20, 43, 82, 101, 102, 142, 173], and **data pruning** [9, 44].

Intermittent local model update. This strategy relaxes the strict synchronization between the client and server, eliminating the necessity to exchange activations and gradients at every training iteration. Instead of continuous communication, these methods allow the system to skip transmission steps based on specific criteria. MESL [3] employs an adaptive threshold mechanism during backward propagation, where the server withholds gradient transmission to the client if the current loss falls below a dynamic threshold. In contrast, CSE-FSL [101] focuses on the forward pass by aggregating local updates for h batches, thereby significantly reducing the frequency of smashed data transmission.

Local loss updates. LocSplitFed [43] pioneered the acceleration of federated split learning through locally generated losses. By employing a lightweight auxiliary network, it enables clients to update their local models using locally generated losses without waiting for back-propagated gradients from the server, thereby achieving parallel training. This eliminates the transmission of back-gradients, drastically cutting down latency and bandwidth consumption. However, the lack of direct server supervision in prior auxiliary models often leads to inaccurate local gradients and performance degradation. Recent studies, such as FSL-SAGE [102] and ESL-LEO [82], address this by introducing a periodic alignment strategy where auxiliary models are regularly updated to accurately mimic the server's behavior. This synchronization bridges the gap between local training and global objectives, ensuring a theoretical convergence rate of $O(1/\sqrt{T})$ while preserving communication efficiency.

Data pruning. This strategy entails filtering out redundant data samples on the client side based on specific criteria before they enter the communication-intensive split training or inference pipeline. SFPrompt* [9] implements a metric-based pruning mechanism using the Error L2 Norm (EL2N) score to quantify sample importance. By utilizing local model components to calculate these scores, the framework selectively retains only high-contribution samples for server interaction. This approach effectively filters out uninformative data, significantly reducing the volume of smashed data transmission while maintaining model performance. Extending this to inference, SplitGP [44] minimizes offloading via confidence-based pruning. It utilizes Shannon entropy to quantify uncertainty, processing confident predictions locally

and only transmitting high-entropy samples to the server. This dynamic filtering reserves bandwidth for difficult tasks necessitating server-side generalization.

4.3.3 Gradient aggregation. Traditional SFL and PSL frameworks typically rely on independent point-to-point communication links between the server and each client to transmit gradients. This paradigm causes the downlink communication overhead to scale linearly with the number of clients, often resulting in severe communication bottlenecks. To address this limitation, several works [71, 103, 106] employ gradient aggregation strategies by consolidating gradients from multiple clients at the server. By broadcasting the unified result to all participants simultaneously, these approaches replace resource-intensive individual transmissions with a single efficient broadcast operation, effectively decoupling communication costs from the client population. Specifically, EPSL [83] introduces a novel last-layer gradient aggregation mechanism, wherein the server aggregates gradients computed at the last layer of the server-side model before backpropagation. By incorporating an aggregation ratio and broadcasting the aggregated gradients to all participating clients, the framework effectively minimizes downlink communication latency, as it completely eliminates the linear scaling of transmission payload.

4.4 Model Aggregation Strategies - Step 11#

Compared to PSL, existing SFL frameworks necessitate the explicit synchronization and aggregation of client-side model weights after multiple local iterations. Beyond the computational and communication overheads, this synchronization imposes a rigid barrier. Under system heterogeneity, this leads to the straggler problem, causing significant training delays as high-capacity clients must wait for the slowest devices to synchronize. MHSL [59] modifies the SplitFedV2 architecture by eliminating the aggregation and synchronization of client-side models (W_c), effectively reverting to a PSL-like paradigm. While the resulting performance degradation is negligible for simple tasks and homogeneous settings, this approach suffers from significant accuracy loss when applied to complex datasets or heterogeneous scenarios. To mitigate the resource inefficiency of synchronous client model aggregation, where fast devices idle waiting for stragglers, DT-ASFL [127] employs an asynchronous aggregation strategy. This enables immediate processing of partial updates, allowing fast devices to train continuously. Furthermore, a "step-aware" mechanism addresses model staleness by weighting updates based on a delay variable; models derived from newer global parameters are assigned higher influence compared to those based on older versions. Similarly, SFL-LEO [142] proposes a staleness-aware asynchronous strategy to handle intermittent connectivity. It mitigates staleness through a dual mechanism: a soft penalty factor that progressively reduces the aggregation weight of lagged updates based on their delay extent, and a hard cutoff threshold that completely discards excessively stale gradients to prevent model divergence. In contrast, SAFSL [2] adopts a semi-asynchronous protocol governed by dynamic time windows. By aggregating only updates completed within the window and allowing stragglers to continue training uninterrupted for future rounds, it effectively eliminates synchronization delays. Furthermore, it employs participation-interval-based weighting to balance contributions and prevent bias toward faster, more frequent devices.

4.5 Server–Client Collaborative Optimization

Traditional SL paradigms impose a strict serial dependency between the client and the server, where the client's forward propagation, data transmission, and the server's forward propagation must be executed sequentially. This "stop-and-wait" interaction mechanism causes computational nodes to remain idle for prolonged periods during communication, severely constraining system resource utilization and training throughput. To circumvent this bottleneck, Server-Client

Collaborative Optimization aims to reconstruct the interaction protocol between the edge and the cloud. By employing strategies such as asynchronous training [17, 116, 155] and parallel training [37, 46, 96, 172], these approaches effectively achieve the decoupling and overlapping of computation and communication.

4.5.1 Asynchronous training strategies. Departing from strict synchronization, AsyncSL [17] implements a loss-based asynchronous update mechanism that dynamically adjusts the synchronization frequency between the client and server. By evaluating the training loss drop against a preset threshold, the framework determines whether to execute a full update or shift to an asynchronous 'silent' mode. This allows the server to continue training using stale activations while the client skips gradient reception and local updates, thereby achieving substantial reductions in both communication overhead and overall training latency. StreamSL* [116] achieves full computation-communication overlap by freezing client-side parameters, restricting local operations to the forward pass. This allows the client to asynchronously transmit intermediate features and labels, immediately processing subsequent batches without awaiting gradient returns. GAS [155] resolves the straggler issue in SFL by decoupling server and client operations via a dual-buffer mechanism. This architecture enables clients to upload data at their own pace while the server executes continuous, non-blocking updates. To mitigate the resulting bias where faster clients dominate the training, the server dynamically synthesizes virtual activations from historical distributions to balance the input stream.

4.5.2 Parallel training strategies. To address the resource under-utilization caused by the sequential execution of computation and communication, PiPar [172] introduces a pipeline parallelism framework that subdivides batches into mini-batches. This mechanism enables the concurrent overlapping of device-side and server-side propagation with the transmission of activations and gradients across mini-batches, thereby effectively minimizing idle time and significantly accelerating the training process. To further enhance training efficiency under resource constraints, C^2P^2SL [85] formulates a joint optimization problem aimed at minimizing the "bubble rate." By leveraging an Alternating Optimization (AO) algorithm, it jointly optimizes the model split point, micro-batch number, batch size, and time slot allocation to mitigate the synchronization latency caused by device heterogeneity. Different from the aforementioned pipeline strategies, Hourglass [46] targets the system scalability bottlenecks inherent in server-side resource management. It proposes a resource-multiplexed data parallelism architecture that decouples the maintenance of server-side model partitions from the massive number of clients. By mapping clients to a limited pool of GPU-resident models, Hourglass effectively eliminates the significant computation and storage overheads caused by frequent model context switching.

4.6 Global Joint Optimization

In the context of Split-based LLM-FT, achieving optimal training performance necessitates the intricate orchestration of heterogeneous resources, including client-server computational power and dynamic communication bandwidth. This process is governed by deeply coupled decision variables. For instance, the selection of the model split point dictates the distribution of computational loads and local memory consumption, which in turn acts as a strict constraint on the feasible batch size, a parameter that directly governs transmission throughput. Consequently, optimizing any single variable in isolation fails to resolve the systemic trade-offs between local resource constraints and global training efficiency. To address this, global joint optimization formulates the fine-tuning process as a constrained optimization problem, jointly adjusting parameters such as the cut layer, batch size, and update frequency to minimize objectives like training latency and energy consumption, as comprehensively summarized in Table 3.

Table 3. Global joint optimization methods. The "Sys. Hetero." column denotes consideration of system heterogeneity across clients. A checkmark (✓) indicates the variable is optimized. Comm. Res.: Communication Resources; Comp. Res.: Computation Resources.

Method	Year	Split Mode	Framework	Optimization Objective	Sys. Hetero.	Decision Variables					Solution Method
						Cut Layer	Batch Size	Update Freq.	Comm. Res.	Comp. Res.	
CPSL [144]	2023	Two-part	GSFL	Training Latency	✓	✓			✓		Two-timescale (SAA + Gibbs Sampling)
U-PSL [95]	2023	Three-part	PSL	Training Latency		✓				✓	LSCRA(Exhaustive + Bisection)
JSFL [150]	2024	Two-part	SFL	Training Latency		✓			✓		Alternating Optimization
EPSL [83]	2024	Two-part	PSL	Training Latency		✓			✓		Block Coordinate Descent
AdaSFL [74]	2024	Two-part	SFL	Training Latency	✓		✓	✓			Linear Programming
MergeSFL [72]	2024	Two-part	SFL	Sync. Wait Time	✓		✓				Genetic Algorithm
EdgeSplit [168]	2024	Two-part	SFL	Training Latency	✓	✓			✓		Alternating Algorithm
ASL [67]	2024	Two-part	SL	Training Latency + Energy Constraints		✓				✓	Lyapunov Optimization + Two-layer Optimization
PASFL* [19]	2025	Two-part	SFL	Latency + Server Energy + Privacy	✓	✓			✓		BCD-based Iterative Algorithm
SAFSL [2]	2025	Two-part	SFL	Training Latency	✓	✓			✓	✓	Lyapunov (Online Drift-Plus-Penalty)
SFL-LLM* [176]	2025	Two-part	SFL	Training Latency	✓	✓			✓		BCD-based Iterative Algorithm
CARD* [66]	2025	Two-part	SL	Latency + Server Energy		✓				✓	Problem Decomposition / Convex Optimization
AdaptSFL [79]	2025	Two-part	SFL	Training Latency	✓	✓		✓			Block Coordinate Descent
SFL-GA [17]	2025	Two-part	SFL	Latency + Convergence	✓	✓			✓	✓	DDQN + CVX
HASFL [75]	2025	Two-part	SFL	Training Latency	✓	✓	✓				BCD-based Iterative Algorithm
HeteroSFL [128]	2025	Two-part	SFL	Training Latency	✓	✓			✓	✓	SAA + GA; Lagrangian + B&B
SFT-LLM* [170]	2025	Two-part	SFL	Training Latency		✓			✓		Decomposition + Augmented Lagrangian + SQP

5 Privacy and Security in Split-based LLM-FT

Despite its architectural advantages for resource-constrained clients and inherent data locality, Split Learning (SL) remains susceptible to various security and privacy risks. While the SL paradigm mitigates the direct exposure of raw inputs and labels by transmitting only intermediate activations—commonly referred to as smashed data—and gradients, these variables still carry significant sensitive features that can be exploited. Recent studies have demonstrated that malicious actors can leverage these intermediate representations to reconstruct private information through methodologies such as model inversion and label inference attacks, or otherwise compromise the integrity of the training process. Accordingly, this section provides a systematic overview of the security landscape in split-based LLM-FT by first categorizing and introducing prominent attack methodologies targeting SL architectures, with a specific focus on those applicable to Large Language Models, and subsequently summarizing the existing defense strategies designed to mitigate these threats and protect the privacy of collaborative fine-tuning. Fig. 6. presents a systematic taxonomy of these attacks and defenses in SL.

5.1 Attacks on Split-based LLM-FT

The security vulnerabilities in Split-based LLM-FT primarily arise from the exchange of intermediate activations, gradients, and model states, which inadvertently expose a broad attack surface, as these high-dimensional representations often retain potent semantic features. Due to the inherent nature of Transformer architectures, such as residual connections that facilitate the preservation of input information throughout deep layers, malicious actors can reconstruct sensitive information including original text sequences and proprietary labels without direct access to raw data. This subsection categorizes the prevailing attack vectors and introduces their underlying technical mechanisms. Notably, methods marked with an asterisk (*) represent attack and defense mechanisms designed for or tested on LLMs. Unmarked methods represent conventional security approaches whose core insights are theoretically transferable to LLM.

5.1.1 Smashed data inversion attacks. During forward propagation, attackers can leverage transmitted smashed data to reconstruct original inputs through model inversion attacks. These threats are categorized into two types based on the

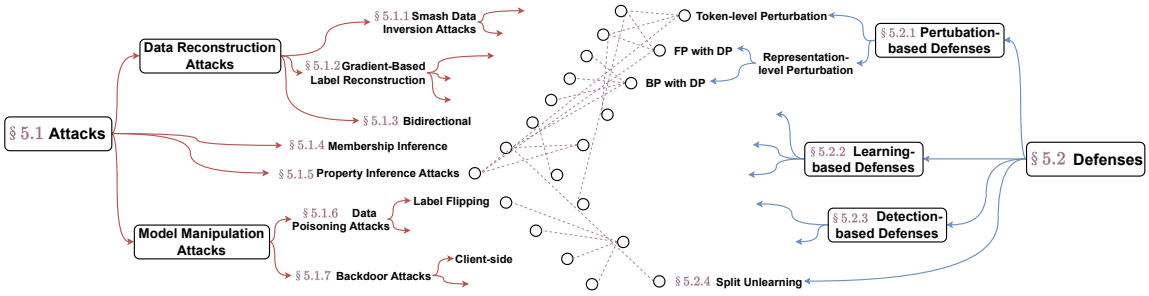


Fig. 6. Taxonomy of attacks and defenses in split-based LLM-FT.

adversary's adherence to the training protocol: (1) active attacks, in which the adversary deliberately manipulates the training procedure or violates established protocols to facilitate more effective information extraction; and (2) passive attacks, where the attacker strictly adheres to the SL protocol by honestly computing and returning gradients without tampering with the training process or model update logic.

Active attacks. In an active attack, a malicious server hijacks or tampers with the gradients backpropagated to the client. The objective is to intentionally guide the client-side model toward specific behaviors or feature representations that expose sensitive information, thereby significantly enhancing the server's ability to reconstruct private data. A prominent example of an active attack is the Feature-Space Hijacking Attack (FSHA) [38, 108]. In this scheme, a malicious server hijacks the training process by providing forged gradients generated from an internal discriminator. These adversarial gradients intentionally guide the client-side model to map its private data into a target, invertible feature space. Once this alignment is achieved, the server utilizes a locally pre-trained decoder to reconstruct the original inputs from the transmitted smashed data with high fidelity. This attack effectively nullifies the privacy-preserving properties of the split learning framework by forcing the client model into an insecure functional state. Despite its potency, FSHA inherently decouples the training process from the intended main task, leaving it vulnerable to detection mechanisms that probe gradient sensitivity toward mislabeled or "fake" samples. To mitigate this vulnerability, recent advancements focus on enhancing the stealth and task-alignment of hijacking attacks. SIA [161], for instance, integrates a server-side main task model that simultaneously disguises the hijacking process and supervises the client-side model. This dual-purpose architecture ensures the extraction of semantic-rich features, rendering the attack far less conspicuous than methods relying solely on adversarial loss.Building on this stealth-oriented logic, SplitSpy [33] addresses the threat of gradient scrutiny by employing a "Legitimate Shadow Model" to pinpoint "fake" samples via their characteristically high prediction errors. By designating the top λ percent of high-loss samples as decoys, SplitSpy implements a differential gradient strategy: it returns legitimate gradients for detected decoys to mimic honest behavior while surreptitiously injecting malicious hijacking signals into real samples. This selective execution allows the adversary to achieve high-fidelity data reconstruction while effectively bypassing sophisticated detection protocols.

Passive attacks. In contrast, passive attacks operate under the honest-but-curious threat model. In this paradigm, the adversary attempts to infer private training instances solely from the received smashed data while strictly adhering to the split learning protocol. This adherence requires the faithful calculation and transmission of gradients without tampering with the training process or the model update logic. Because these attacks do not disrupt the primary training objective, they exhibit high stealthiness and can effectively circumvent active defense mechanisms designed to identify gradient anomalies. Existing literature typically classifies passive inversion strategies into two distinct paradigms: *optimization-based* and *learning-based* approaches [48].

(1) Optimization-based approaches [27, 68]. These methods model data reconstruction as an iterative optimization problem, aiming to find an input that matches the observed smashed data or gradients. Specifically, UnSplit [27] models the attack as an optimization problem and solves it using coordinate gradient descent. The attacker alternately updates the parameters of the reconstructed input and the proxy model. This method allows the server to simultaneously steal the client's model functionality without accessing auxiliary public datasets or query functions, and recovers the private input with high fidelity by minimizing mean squared error and total variation loss. Similarly, GLASS [68] reformulates the attack as a latent space search within a pre-trained StyleGAN. It reconstructs the private input by optimizing latent codes to minimize the distance between the intermediate features of the generated and target data. This two stage strategy explores the entangled space to escape local minima and refines the result in the disentangled space. To address optimization challenges at deeper layers, GLASS++ [68] improves this method by employing an encoder to map features directly to the latent space. This provides a robust initialization for the latent search, preventing the optimization from getting trapped in local minima and ensuring stability when spatial information is limited.
(2) Learning-based approaches [36, 94, 152, 166, 167, 182]. Unlike optimization methods, learning-based approaches typically involve constructing a shadow model or pseudo-client trained on an auxiliary dataset to learn an inverse mapping from smashed data back to the input space. PCAT [36] trains a pseudo-client model on a small auxiliary dataset by minimizing the downstream task loss (e.g., Cross-Entropy) calculated via the server model. Based on this simulation, the server trains a reverse mapping network to reconstruct original inputs from intercepted smashed data. Furthermore, PCAT adopts a "stealing while training" strategy that compels the pseudo-client to track the real client's learning trajectory. This prevents convergence to local optima with mismatched internal features, significantly enhancing both functionality stealing and data reconstruction. However, when victim clients employ label transformation defenses (e.g., mapping real labels to fake or encoded ones), Basic PCAT fails as the server's top model learns incorrect mappings, misleading the pseudo-client. Improved PCAT [167] addresses this by introducing a pseudo-top Model to substitute the compromised real top layer, effectively eliminating dependency on the server's original task objective. By leveraging the shared intermediate model for feature extraction and employing a multi-model parallel training strategy to identify the best match, Improved PCAT successfully steals authentic functionality despite label tampering, significantly enhancing attack robustness. MAERA [166] contends that relying exclusively on task-oriented objectives (e.g., cross-entropy) is insufficient for high-fidelity data reconstruction. While such methods may achieve functional equivalence, they suffer from a significant "feature space gap," yielding reconstructed data lacking original semantic details. To bridge this disparity, MAERA proposes a GAN-based framework employing a hybrid loss function. Specifically, the discriminator minimizes the divergence between shadow and authentic features, ensuring the shadow model's output statistically aligns with the real client's latent space. Other approaches adopt similar principles by incorporating adversarial losses during the reconstruction phase to enhance the fidelity of the recovered data. For instance, FSA [94] employs a cycle-consistent generative adversarial network to align smashed data with a feature space defined by a pre-trained autoencoder. Similarly, FORA [152] leverages domain discriminator loss combined with multi-kernel maximum mean discrepancy to project and align the output features of the pseudo-client and the real client within a shared latent space. SDAR [182] refines this using dual adversarial discriminators to respectively align the simulator's intermediate representations with authentic client outputs, and the decoder's reconstructions with real natural sample distributions.

5.1.2 Gradient-based label reconstruction. In u-shaped SL, even though clients keep labels locally, the gradients sent to the server still carry hidden label information. This occurs because gradients are calculated based on the difference between the model's prediction and the actual label. As a result, a malicious server can analyze these gradients

Table 4. Summary of smashed data inversion attacks on Split-based LLM-FT. (✓: required/known, ×: not required/unknown)

Method	Attack Type	Reconstruction Paradigm	Attack Phase	Attacker's Capability		
				Client Arch.	Client Weights	Aux. Dataset
FSHA [38]	Active	Learning-based	Training	×	×	✓
SIA [161]	Active	Learning-based	Training	×	×	✓
SplitSpy [33]	Active	Learning-based	Training	×	×	✓
UnSplit [27]	Passive	Optimization-based	Inference	✓	×	×
GLASS [68]	Passive	Optimization-based	Inference	✓	✓	✓
GLASS++ [68]	Passive	Hybrid	Inference	✓	✓	✓
PCAT [36]	Passive	Learning-based	Training	×	×	✓
Improved PCAT [167]	Passive	Learning-based	Training	×	×	✓
MAERA [166]	Passive	Learning-based	After convergence	×	×	✓
FSA [94]	Passive	Learning-based	Inference	×	×	✓
FORA [152]	Passive	Learning-based	Training	×	×	✓
SDAR [182]	Passive	Learning-based	Training	×	×	✓

to recover private labels. The risk of such label leakage is further exacerbated in the context of LLMs. Specifically, for Causal Language Modeling (CLM) tasks that utilize autoregressive mechanisms, the supervision signal (i.e., the label) is essentially the input sequence shifted by one token. This implies that in such scenarios, stealing labels is effectively equivalent to reconstructing the private raw input text [12, 91]. Existing gradient-based label reconstruction attacks can be categorized into three streams based on their underlying mechanisms: metric-based gradient analysis, optimization-based gradient matching, and learning-based label inversion.

Metric-based gradient analysis attacks. These attacks exploit the mathematical properties of loss functions. By calculating specific statistical metrics (e.g., gradient norm, distance, similarity) on the gradients or smashed data at the split layer, attackers can directly distinguish ground-truth labels. Li et al. [65] recover private labels by analyzing returning gradients via two methods. The norm-based attack exploits asymmetrical gradient norm distributions, where positive samples generate larger gradients, allowing threshold-based recovery. Meanwhile, the Direction-based Attack leverages the geometric property that same-class gradients form acute angles, enabling reconstruction through cosine similarity and clustering. However, these methods are specifically designed for binary classification tasks and may not be directly applicable to more complex multi-class scenarios. Liu et al. [86] address this with a distance-based attack unifying gradient and smashed data metrics into euclidean distance. Furthermore, they utilize pre-trained models to extract auxiliary baseline features, inferring labels via target-auxiliary distances. In a significant extension, Liu et al. [87] exploit latent grouping characteristics by integrating K-means clustering with the K-M matching algorithm. This method maps latent data clusters to ground-truth labels using limited auxiliary samples, demonstrating superior scalability on complex datasets and enhanced robustness against gradient compression.

Optimization-based gradient matching attacks. This category of attacks models label reconstruction as an iterative optimization task based on gradient matching. The adversary initializes dummy labels and a surrogate model to generate virtual gradients, then iteratively optimizes them by minimizing the discrepancy with the client's real gradients to recover the true labels. UnSplit [27] executes label inference by minimizing the Mean Squared Error (MSE) between the client's actual gradients and a surrogate's proxy gradients across all label candidates. Identifying the closest gradient profile enables perfect label inference. However, this attack is primarily effective when the label owner's local model is shallow (e.g., one layer deep). To overcome this limitation, ExPLoit [61] incorporates label prior and label entropy regularization into its loss function. By enforcing alignment between surrogate predictions and labels, this method avoids poor local minima, maintaining efficacy even against multi-layer networks. Xie et al. [149] demonstrated that by utilizing a training accuracy loss, label inference attacks in split learning can be extended from classification

problems to regression problems. SplitAUM [175] introduces a semi-supervised clustering strategy (e.g., K-Means) to initialize dummy labels with feature distributions matching the ground-truth. This addresses the limitation of random initialization, which often misleads the model. Leveraging the "not-too-far" characteristic of LLM fine-tuning, Chen et al.* [12] proposed directly utilizing the tail layers of the pre-trained model as the surrogate model. By employing the pre-trained weights as a robust prior, this strategy creates a semi-white-box environment that eliminates the need to optimize a randomly initialized surrogate from scratch. Crucially, this setting empowers the adversary to effectively deploy existing Gradient Matching Attacks (GMA), including DLG [181], iDLG [174], TAG* [22], and LAMP* [5], to recover private labels. Consequently, the approach significantly enhances attack accuracy by combining the stability of pre-trained priors with the optimization power of established gradient inversion algorithms.

Learning-based label inversion attacks. Unlike optimization-based methods that rely on per-sample iteration, learning-based attacks aim to functionally complete the split network. The adversary leverages auxiliary data to train a pseudo-tail model that acts as a substitute for the victim's original tail model, thereby restoring the direct mapping from smashed data to private labels. Once this model completion process is finished, the adversary can seamlessly append the pseudo-tail model to the split layer for efficient, end-to-end label inference. Fu et al. [31] proposed a passive inference attack based on model completion, where the adversary appends an inference head to their trained bottom model and fine-tunes it using limited auxiliary labeled data via semi-supervised learning. Once fine-tuned, this completed model is used to directly predict the labels of any target sample based solely on the adversary's local features. They further enhanced this approach with an active attack using a malicious local optimizer, which adaptively scales gradients to induce the global model to rely more heavily on the adversary's features, thereby improving the expressiveness of the bottom model for label inference. To address the overfitting issue where pseudo-tail models memorize auxiliary data instead of learning transferable features, SDAR [182] employs random label flipping. This regularization strategy effectively enhances the model's generalization ability, enabling accurate label inference in u-shaped SL.

5.1.3 Bidirectional reconstruction attacks. Previous attacks typically exploit a single source of information, utilizing either the smashed data from forward propagation or the gradients from backward propagation to recover private labels or input text. However, in the context of Split-based LLM fine-tuning, these two data streams contain complementary semantic information. Furthermore, due to the autoregressive nature of Causal Language Modeling (CLM), the input text and the supervision labels, which represent the next-token prediction, are semantically equivalent as they differ by only a single token shift. Bidirectional reconstruction attacks aim to integrate the leaked information from both forward and backward propagation to achieve higher fidelity in data recovery. This approach effectively overcomes the inherent information bottlenecks associated with unidirectional attacks. Chen et al.* [12] pioneered this direction by proposing Bidirectional Semi-white-box Reconstruction (BiSR), a framework specifically tailored for Split-based private LLM fine-tuning. Grounded in the "not-too-far" assumption, which posits that the parameters of a fine-tuned model remain spatially close to those of the publicly available pre-trained model, BiSR employs a synergistic two-pronged strategy. It integrates the Semi-white-box Inversion Paradigm (SIP), where the adversary leverages pre-trained weights to train an inverse network mapping smashed data back to input embeddings, with a backward gradient matching mechanism to extract token information. BiSR's critical innovation lies in deeply coupling these processes: high-level semantic features from the forward SIP module regularize the backward gradient matching objective. This semantic guidance prevents gradient matching from falling into poor local minima, reconstructing private fine-tuning data with significantly higher accuracy and coherence than unidirectional leakage methods.

5.1.4 Membership inference attacks. Membership Inference Attacks (MIA) are designed to infer whether a specific data record was included in the training dataset of a target model. Traditional MIA operates in a black-box setting, requiring query access to the target model. Attackers train shadow models using the same ML platform or a similar architecture to replicate the target's behavior and infer membership. However, in Split Learning, attackers lack access to the client-side model segment, rendering direct queries impossible. To address this, Chen et al. [14] proposed a novel "Transfer-Inherit" strategy. They substitute the inaccessible client layers with a pre-trained feature extractor and splice it with the accessible server-side layers. By fine-tuning this feature extractor on a shadow dataset, the attacker can effectively mimic the target model's behavior with limited data. This local shadow model is then used to train the attack model, enabling membership inference without querying the actual target system, thus achieving a "query-free" attack. To address the significant prediction misalignment when using shadow model responses to estimate the target model's memory, Yi et al. [159] proposed the Knowledge Transfer Membership Inference Attack (KTMIA), which reconstructs both models by splicing pseudo-clients onto frozen server parameters to ensure aligned membership signals.

5.1.5 Property inference attacks. Property Inference Attacks [98, 108] target the extraction of specific sensitive attributes that are often orthogonal to the primary learning task, such as author identity, political inclination, or underlying emotional sentiments. Compared to full data reconstruction, these attacks typically exhibit lower implementation complexity and greater robustness against existing defense mechanisms. In the context of split learning, an attacker executes this by replacing the original server-side model with a specific property classifier. By leveraging a labeled auxiliary dataset for supervised training, the attacker effectively coerces the client model to align its feature space with representations that explicitly encode the target attribute, thereby enabling the precise inference of private information.

5.1.6 Data poisoning attacks. Data poisoning attacks involve the injection of corrupted or malicious samples into the training dataset during the model's training phase, thereby compromising its integrity or usability. In the context of multi-client Split-based LLM-FT, such as multi-client sequential SL or SFL, one or more malicious clients can manipulate their local training data to significantly degrade the performance of the global model.

Label flipping attacks. Label-flipping attack in Split-based LLM-FT is a form of dirty-label poisoning where malicious clients intentionally manipulate their local training labels to induce erroneous model predictions and degrade the system's overall robustness. Kohankhaki et al. [63] explored label-poisoning threats within multi-client sequential SL, revealing that the integrity of global models is highly sensitive to even minor data manipulation by malicious participants. Gajbhiye et al. [34] analyzed the vulnerability of SFL variants (SFLV1 and SFLV2) to label flipping attacks. While both variants proved susceptible, SFLV2 demonstrated significantly lower robustness. The study also concluded that untargeted attacks cause more severe degradation to the global model than targeted attacks. Ismail and Shukla [165] further expanded the threat landscape of SFL by introducing a novel distance-based poisoning strategy. Unlike previous works that relied on random or fixed label selection, this approach optimizes the attack by selecting target labels that maximize the Euclidean distance between input samples. Their research demonstrated that this distance-oriented method achieves a more potent degradation of model performance than traditional targeted attacks.

Smash data poisoning attacks. While label flipping attacks target the supervision signal directly, they are often directly adapted from Federated Learning (FL) without exploiting the unique architectural characteristics of split-based learning. Wu et al. [145] proposed a novel attack specifically tailored for SFL, named Smash Poisoning, which applies gradient ascent perturbations to the smashed data by leveraging historical gradients. Furthermore, combining this method with Label Poisoning was shown to significantly enhance the attack's potency. Recently, a hybrid attack strategy named MISA [137] was proposed to induce global model misalignment by simultaneously targeting the server-side top

model and the client-side model. This method indirectly poisons the top model through statistically crafted perturbations on smashed data, while employing Thompson Sampling and a Search-and-Locate (SnL) mechanism to optimize the bottom model manipulation as a Min-Sum problem.

5.1.7 Backdoor attacks. Backdoor attacks [39] refer to an adversary implanting a specific pattern or "trigger" into a machine learning model. This allows the trained model to maintain state-of-the-art performance on clean validation samples to remain stealthy while being subtly manipulated to output a specific target class whenever the trigger appears in the input during testing. Traditional backdoor attacks are typically implemented through training set poisoning where a portion of the training data is modified to include the trigger and reassigned to the attacker's target label. In the context of Split-based LLM-FT, these attacks can be categorized into client-side backdoor attacks and server-side backdoor attacks, depending on which party initiates the attack.

Client-side backdoor attacks. In scenarios where the client possesses both raw inputs and ground-truth labels, it can readily deploy traditional backdoor strategies, such as injecting specific triggers into training samples and flipping their labels to implant malicious associations [160]. However, in the context of two-party SL, where data and labels are vertically partitioned, the client holds only the input data. This lack of access to supervision signals makes executing backdoor attacks significantly more challenging, as the adversary cannot directly manipulate the label to establish the trigger-target correlation. Yu et al. [160] propose that attackers can employ label inference attacks to bypass feature-label isolation and identify target samples. Furthermore, by integrating an auxiliary model at the client side to distinguish between clean and backdoor features, the attacker forces the client model to map triggered samples into a distinct, isolated region of the feature space, thereby significantly enhancing the success rate of the backdoor attack. He et al. [47] leverage the Mean-shift algorithm to extract a high-density "trigger vector" from the local embeddings of a small auxiliary set belonging to the target class. By consistently injecting this vector during training, the malicious client forces the top model to establish a strong associative mapping between the trigger and the target label. During inference, the attacker can hijack the classification of any sample by replacing its local embedding with this trigger, effectively overriding host-side features to trigger the desired misclassification. Bai et al. [4] propose VILLAIN, a framework utilizing gradient-based label inference. By swapping sample embeddings with a target seed and monitoring gradient magnitude, the attacker identifies target labels. To enhance stealth, a semi-supervised classifier restricts this inference to selected candidates. VILLAIN then injects additive triggers into high-variance features, employing trigger randomization and learning rate adjustment to ensure robust backdoor implantation. PPoison [88] injects a backdoor by introducing a pluggable trigger layer that directly transforms the smashed data output by the local model, optimized via a local surrogate classifier. In normal mode, the trigger layer remains inactive, transmitting original features to ensure accurate predictions; in attack mode, the layer is activated to convert embeddings from arbitrary inputs into poisoned representations, compelling the server-side model to misclassify them as a preset target label.

Server-side backdoor attacks. In the server-side attack model, due to the honesty of the clients, attackers can neither access the raw input nor inject triggers, leading to significant structural barriers for backdoor attacks. Tajalli et al. [129] proposed the first threat model for server-side backdoor attacks in SL. The authors introduced two methods, the surrogate client and the injector autoencoder, to inject backdoors without access to client inputs. However, their findings indicate that these attacks are only viable on simple datasets and largely fail as data complexity increases. Targeting scenarios where the server holds the labels, IBAS leverages a small amount of public data to train a "parrot model" that approximates the client's feature space. During the forward pass of training, the server replaces a subset of the client's uploaded intermediate activations with poisoned ones generated by the parrot model. By modifying the

corresponding labels and backpropagating the gradients, IBAS [146] effectively guides the client model to embed a backdoor while preserving the performance of the main task. In scenarios where the server does not possess ground truth labels, executing server-side attacks becomes significantly more challenging Yu et al. [160] leveraged a shadow model trained on domain-consistent data to pre-simulate backdoor injection. By employing a GAN-based discriminator, they forced the alignment of the client's feature space with that of the shadow model, thereby endowing the client with indirect backdoor extraction capabilities. Additionally, an auxiliary model maximizes the separability of backdoor features to prevent them from being diluted during training. However, to mitigate the dependency on large-scale public datasets and improve main task alignment, SFI [162] further deconstructs the attack into three distinct stages (Steal, Finetune, and Implant). This staged design allows the model to first learn the feature space before injecting the backdoor, exerting finer control over the attack process. This approach ensures stability and effectiveness, thereby improving the attack's accuracy in complex scenarios. Dullahan [113] is a stealthy post-training attack strategy designed for the without-label-sharing architecture, diverging from previous methods by refraining from manipulating intermediate parameters (e.g., gradients) during the collaborative training phase. To overcome the lack of label access, the method constructs a surrogate model to approximate the client's feature output and employs K-means clustering to identify a "target embedding anchor" as a substitute for the inaccessible target label. The backdoor is subsequently injected by fine-tuning the server network to map selected trigger embeddings to this anchor, thereby achieving high stealthiness against gradient-based detection mechanisms.

5.2 Defenses on Split-based LLM-FT

To mitigate the security and privacy risks inherent in the collaborative training process, research has increasingly focused on developing robust defense mechanisms for Split-based LLM-FT. The primary objective of these strategies is to obfuscate or protect the sensitive features embedded within intermediate representations and gradients, thereby preventing adversaries from successfully executing inversion or inference attacks. However, designing effective defenses requires navigating a complex trade-off between the level of privacy protection, system execution efficiency, and the final model performance. This subsection categorizes the prevailing defense strategies into three primary paradigms based on their underlying protection mechanisms: perturbation-based, learning-based, and cryptographic-based defenses.

5.2.1 Perturbation-based defenses. Perturbation-based defenses involve introducing controlled randomness into the training pipeline to obfuscate sensitive information while maintaining as much model utility as possible. These methods are frequently grounded in the principles of Differential Privacy (DP) [25], providing a quantifiable and mathematically rigorous trade-off between privacy protection and fine-tuning accuracy. In the context of Split-based LLM-FT, these strategies can be broadly categorized based on the granularity of noise injection: token-level perturbation, which operates on the discrete input sequence, and representation-level perturbation, which injects noise into continuous vectors such as embeddings, activations, and gradients.

Token-level perturbation (discrete perturbation). Token-level Perturbation targets the discrete input space of LLMs by substituting sensitive tokens within the input sequence with generic or alternative replacements. By perturbing the sequence while preserving essential semantic structures, this strategy mitigates the risk of private text reconstruction from the earliest stage of the training pipeline. Embedding dχ-Privacy (dχP) [10] injects Laplacian noise into embedding outputs and maps the perturbed results to their nearest valid neighbors in the embedding space to preserve semantic coherence while providing privacy protection. To mitigate utility degradation from uniform noise, SAP* [123] employs Contributing-Token-Identification (CTI) to identify task-critical tokens and adaptively allocate privacy budgets by

reducing perturbations on important tokens, thereby achieving a superior privacy-utility trade-off. SanText* [164] generates natural, high-utility text satisfying local differential privacy by using probability sampling based on word embedding Euclidean distances to replace original tokens with semantically similar substitutes. However, SanText uses the entire vocabulary as its sampling candidate set, which can easily introduce semantically irrelevant interference and lead to redundant privacy costs when privacy budgets are tight. In contrast, CusText* [15] effectively eliminates irrelevant terms by allocating a smaller, semantically highly relevant customized output set to each token, significantly improving the utility of anonymized data while reducing privacy overhead. Furthermore, Rantext* [132] introduces a random adjacency list mechanism that leverages the Laplace distribution to dynamically determine the list size for each token, thereby increasing the difficulty for adversaries to reconstruct the raw tokens. Cape* [141] integrates a hybrid utility function combining Euclidean token distances with client-side contextual logits and employs a bucketized sampling mechanism to mitigate the large-vocabulary long-tail phenomenon, effectively suppressing low-utility candidate probabilities to enhance semantic coherence and quality while ensuring differential privacy.

Representation-level perturbation (continuous noise injection). This approach involves injecting continuous noise such as Gaussian or Laplacian noise into high-dimensional intermediate values including embedding vectors, smashed data (activations), and backpropagated gradients.

(1) Forward Propagation with DP Noise. [23, 55, 97, 107, 111, 139] This strategy injects noise into intermediate values including embedding vectors and smashed data during forward propagation before they are transmitted from the client to the server. Pan et al. [107] mitigate information leakage by injecting Laplacian noise into embedding vectors. To overcome the utility degradation inherent in traditional perturbation mechanisms, SnD* [97] implements d_χ-privacy while deploying a pre-trained denoising module on the client side. This architecture enables localized correction of the perturbed inference results, thereby maximizing the preservation of model utility. DP-forward *[23] schemes incorporate noise into activation values. These privacy-preserving techniques are designed to prevent adversaries from inferring original inputs from perturbed smashed data, thereby ensuring robust privacy protection throughout the forward propagation process. To address the utility degradation caused by uniform token perturbation in DP-forward, Wang et al.* [139] propose SLDP-FT. This framework selectively injects noise only into identified sensitive tokens. Furthermore, it dynamically calibrates the noise scale across hidden layers using a privacy forward weight, achieving a superior privacy-utility trade-off.

(2) Backward propagation with DP noise. [114, 143, 157, 157] This strategy injects noise during the backward pass to protect sensitive information within transmitted gradients. It involves adding Gaussian or Laplacian noise to model gradients, cut layer gradients, and labels to effectively mitigate the risk of label reconstruction attacks. Gawron et al. [38] evaluated the robustness of DP-SGD-based client defense in split learning by applying gradient clipping and Gaussian noise to client-side model to suppress FSHA reconstruction accuracy and delay privacy leakage. GradPerturb [157] provides provable differential privacy by injecting directional noise at the final hidden layer along the vector difference between the true and alternative label gradients. This targeted approach ensures rigorous privacy guarantees while maintaining high model utility. Marvell [65] mitigates label leakage by strategically perturbing cut-layer gradients, employing an optimized noise structure that minimizes the symmetric KL divergence between class-conditional gradient distributions. Several research [114, 143] efforts secure the backward pass through the clipping of cut layer gradients and the subsequent injection of DP noise. By adding Laplace or Gaussian noise to these clipped gradients, these methods effectively thwart label inference attacks while theoretically maintaining the convergence of model training. To further refine this approach, ProjPert [32] formulates noise injection as a mathematical projection problem under privacy constraints, computing the optimal noise to minimize utility loss measured by Wasserstein distance. Label DP [114, 157]

is implemented using a randomized response algorithm that randomly flips labels during training based on a specific probability, thereby preventing an attacker from perfectly reconstructing users' private labels. Building upon this, PSLF introduces a Flipped Multi-Label Generation mechanism and two separate top sub-models, training one on flipped multi-labels to thwart label leakage and the other on true binary labels to boost prediction accuracy.

5.2.2 Learning-based defenses. Learning-based defenses integrate privacy constraints directly into the optimization process via auxiliary training objectives or specialized loss functions. Unlike perturbation-based approaches that treat noise injection as a post-processing step, these strategies optimize internal representations or gradients to proactively minimize the leakage of sensitive information. NoPeek* [124, 135] reduces the risk of sensitive data leakage by incorporating distance correlation as a regularization term in the loss function to minimize the statistical dependency and achieve decorrelation between raw input and smashed data. Building upon the core objective of dependency minimization, Zou et al. [183] introduced a Mutual Information Regularization [140] term during training to effectively resist data reconstruction attacks and backdoor attacks by limiting the mutual information between private raw data and intermediate outputs while maintaining model utility. Eguard* [90] utilizes a projection network to migrate original embeddings into a secured space. It decouples the sensitive correlation between raw text and vectors through mutual information optimization, while a functionality preservation module ensures downstream task performance remains highly consistent with the original embeddings. TextObfuscator* [179] identifies task-related prototypes through clustering and utilizes clustering and distance losses to group functionally similar representations while maintaining inter-prototype separation. By applying random perturbations within these clusters, it renders word vectors indistinguishable to protect privacy while preserving their original task functionality. ResSFL [64] mitigates model inversion attacks by pre-training a resistant feature extractor via attacker-aware training and bottleneck layers, which is then transferred to initialize client models, maintaining high accuracy while reducing computational overhead and avoiding early-epoch vulnerabilities. DualGuard* [91] introduces a bidirectional defense for split-based LLM fine-tuning that uses local warm-up and global retention to transform the parameter space. By increasing the dissimilarity between client-side and pre-trained models, it successfully defends against both forward activation-based and backward gradient-based reconstruction attacks while preserving model utility.

5.2.3 Detection-based defenses. Detection-based defenses focus on monitoring the training process to identify and respond to malicious activities in real-time.

Gradient attribute monitoring. This category of defense aims to prevent a malicious server from providing forged gradients to hijack the client's feature space to reconstruct original inputs. A representative approach is SplitGuard [26], which detects training-hijacking attacks by leveraging the distinctively larger gradient angles produced when clients intermittently process "fake batches" with randomized labels. By analyzing these gradient discrepancies, SplitGuard enables clients to identify hijacked models and halt training before significant privacy leakage occurs. However, SplitGuard is vulnerable to the SplitSpy attack [33], where a malicious server leverages a shadow model to identify fake samples and generate deceptive gradients. To address this, Fu et al. proposed Gradients Scrutinizer [33], a passive monitoring mechanism that distinguishes honest training from hijacking by analyzing intrinsic gradient attributes: set gap, fitting Error, and overlapping ratio. This approach is robust against evasion and preserves model accuracy by avoiding the need for active sample injection. SplitOut [28] uses the Local Outlier Factor (LOF) algorithm to monitor server-returned gradients in real-time, identifying malicious training-hijacking as outliers to intercept and terminate the attack at an early stage.

Model parameter monitoring. This strategy is designed to identify and prevent malicious clients from conducting backdoor attacks, where attackers attempt to implant hidden triggers into the model's parameters during the collaborative training process. SafeSplit [118] determines whether a client has initiated an attack by monitoring the parameter updates of the server-side backbone before and after each client's training. Specifically, it utilizes the Discrete Cosine Transform (DCT) to monitor abnormal fluctuations of model parameters in the frequency domain as static analysis. For dynamic analysis, it introduces a rotational distance metric to capture directional mutations and angular velocity anomalies of the update trajectory in multi-dimensional space. If a client is discovered to be non-benign, the server determines that the training process is non-smooth and malicious. Subsequently, it activates a rollback mechanism.

5.2.4 Split Unlearning. Machine unlearning [7] is essential for removing private or harmful data from trained models. However, the deep client-server coupling in traditional SL hinders efficient unlearning. To address this, SPLITWIPER [58] introduces a novel "one-way-one-off" propagation scheme. Clients freeze their weights after lightweight local pre-training and transmit intermediate outputs to the server only once. This decouples the network, satisfying the "Isolated" requirement of the SISA unlearning framework. Consequently, unlearning requests only require the target client to retrain, leaving others unaffected and enabling independent unlearning in multi-client SL architectures.

5.2.5 Other Defenses. **Homomorphic encryption.** Homomorphic Encryption (HE) ensures zero privacy leakage by executing computations entirely on ciphertexts. Although HE has been actively explored in standard Split Learning SL [60, 70, 109] and LLM inference [16, 49, 92], its prohibitive computational and communication overheads currently preclude its practical application in split-based LLM fine-tuning.

Protocol modifications. Several studies preserve privacy by modifying the SL protocol. 3-Tier SL [1] introduces a "Three-Tier Split Learning" architecture, adding an "Aggregator" as an intermediary between clients and the central server. Clients send only their activations to the aggregator, which averages them before forwarding them to the server, successfully preventing direct raw data leakage. However, this inevitably compromises model performance. P-SL [112] is designed to prevent colluding malicious clients and servers from using shared local weights to reconstruct victims' private data via model inversion attacks. Consequently, P-SL fundamentally prohibits local weight sharing among clients, resulting in a unique local model for each client (similar to parallel split learning). Yet, because its primary focus is preventing client-server collusion, it overlooks the risk of a malicious server directly executing smashed data inversion attacks using methods like those mentioned above (Section subsubsection 5.1.1).

6 Conclusion

This survey provides a comprehensive review of recent advancements in integrating Split Learning with LLM Fine-Tuning (Split-based LLM-FT) , systematically exploring model-level optimizations, system-level performance enhancements, and privacy-preserving mechanisms. These components are abstracted into a cohesive, highly granular unified training pipeline : system-level optimizations, such as communication reduction and server scheduling, resolve critical bottlenecks and resource imbalances to ensure scalable execution ; model-level strategies tackle data heterogeneity, server-client update imbalances, and catastrophic forgetting to guarantee robust convergence and global accuracy ; and advanced privacy and security defenses safeguard sensitive intermediate representations against a diverse landscape of inversion, inference, and manipulation attacks. By systematically addressing these fundamental challenges across efficiency, performance, and security , this survey offers valuable insights for researchers and practitioners , paving the way for accessible, resource-efficient, and secure collaborative AI solutions that unlock the full potential of customized LLMs.

References

[1] Arwa Alromih, John A. Clark, and Prosanta Gope. 2022. Privacy-Aware Split Learning Based Energy Theft Detection for Smart Grids. In *Information and Communications Security: 24th International Conference, ICICS 2022, Canterbury, UK, September 5–8, 2022, Proceedings* (Canterbury, United Kingdom). Springer-Verlag, Berlin, Heidelberg, 281–300.

[2] Huiqing Ao, Hui Tian, Wanli Ni, Gaofeng Nie, and Dusit Niyato. 2025. Semi-Asynchronous Federated Split Learning for Computing-Limited Devices in Wireless Networks. *IEEE Transactions on Wireless Communications* 24, 6 (2025), 5196–5212.

[3] Ahmad Ayad, Melvin Renner, and Anke Schmeink. 2021. Improving the communication and computation efficiency of split learning for iot applications. In *Proceedings of the 2021 IEEE Global Communications Conference* (Madrid, Spain). IEEE, Piscataway, NJ, USA, 01–06.

[4] Yijie Bai, Yanjiao Chen, Hanlei Zhang, Wenyuan Xu, Haiqin Weng, and Dou Goodman. 2023. VILLAIN: backdoor attacks against vertical split learning. In *Proceedings of the 32nd USENIX Conference on Security Symposium*. USENIX Association, Anaheim, CA, USA.

[5] Mislav Balunovic, Dimitar Dimitrov, Nikola Jovanović, and Martin Vechev. 2022. LAMP: Extracting Text from Gradients with Language Model Priors. In *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh (Eds.), Vol. 35. Curran Associates, Inc., 7641–7654.

[6] Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Maksim Riabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. 2023. Petals: Collaborative Inference and Fine-tuning of Large Models. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*. Association for Computational Linguistics, Toronto, Canada, 558–568.

[7] Lucas Bourtoule, Varun Chandrasekaran, Christopher A Choquette-Choo, Hengrui Jia, Adelin Travers, Baiwu Zhang, David Lie, and Nicolas Papernot. 2021. Machine unlearning. In *2021 IEEE symposium on security and privacy (SP)*. IEEE, 141–159.

[8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. *Advances in neural information processing systems* 33 (2020), 1877–1901.

[9] Linxiao Cao, Yifei Zhu, and Wei Gong. 2024. Sfprompt: Communication-efficient split federated fine-tuning for large pre-trained models over resource-limited devices. arXiv:2407.17533 https://arxiv.org/abs/2407.17533

[10] Konstantinos Chatzikokolakis, Miguel E Andrés, Nicolás Emilio Bordenabe, and Catuscia Palamidessi. 2013. Broadening the scope of differential privacy using metrics. In *international symposium on privacy enhancing technologies symposium*. Springer Berlin Heidelberg, Berlin, Heidelberg.

[11] Manisha Chawla, Gagan Raj Gupta, Shreyas Gaddam, and Manas Wadhwa. 2024. Beyond Federated Learning for IoT: Efficient Split Learning With Caching and Model Customization. *IEEE Internet of Things Journal* 11, 20 (2024), 32617–32630.

[12] Guanzhong Chen, Zhenghan Qin, Mingxin Yang, Yajie Zhou, Tao Fan, Tianyu Du, and Zenglin Xu. 2024. Unveiling the vulnerability of private fine-tuning in split-based frameworks for large language models: A bidirectionally enhanced attack. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA, 2904–2918.

[13] Haitian Chen, Xuebin Chen, Lulu Peng, and Yuntian Bai. 2023. Personalized fair split learning for resource-constrained Internet of Things. *Sensors* 24, 1 (2023), 88.

[14] Hanxiao Chen, Hongwei Li, Guishan Dong, Meng Hao, Guowen Xu, Xiaoming Huang, and Zhe Liu. 2020. Practical membership inference attack against collaborative inference in industrial IoT. *IEEE Transactions on Industrial Informatics* 18, 1 (2020), 477–487.

[15] Sai Chen, Fengran Mo, Yanhao Wang, Cen Chen, Jian-Yun Nie, Chengyu Wang, and Jamie Cui. 2023. A customized text sanitization mechanism with differential privacy. In *Findings of the Association for Computational Linguistics*. Toronto, Canada, 5747–5758.

[16] Tianyu Chen, Hangbo Bao, Shaohan Huang, Li Dong, Binxing Jiao, Daxin Jiang, Haoyi Zhou, Jianxin Li, and Furu Wei. 2022. The-x: Privacy-preserving transformer inference with homomorphic encryption. In *Findings of the association for computational linguistics: ACL 2022*. Association for Computational Linguistics, Dublin, Ireland, 3510–3520.

[17] Xing Chen, Jingtao Li, and Chaitali Chakrabarti. 2021. Communication and computation reduction for split learning using asynchronous training. In *2021 IEEE Workshop on Signal Processing Systems (SiPS)*. IEEE, 76–81.

[18] Xiaopei Chen, Liang Li, Fei Ji, and Wen Wu. 2025. Memory-efficient split federated learning for llm fine-tuning on heterogeneous mobile devices. In *IEEE INFOCOM 2025-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 1–6.

[19] Xiaopei Chen, Wen Wu, Fei Ji, Yongguang Lu, and Liang Li. 2025. Privacy-aware split federated learning for LLM fine-tuning over internet of things. *IEEE Internet of Things Journal* (2025).

[20] Ayush Chopra, Surya Kant Sahu, Abhishek Singh, Abhinav Java, Praneeth Vepakomma, Mohammad Mohammadi Amiri, and Ramesh Raskar. 2023. Adaptive split learning. In *Federated Learning Systems (FLSys) Workshop@ MLSys 2023*.

[21] Haiyu Deng, Yanna Jiang, Guangsheng Yu, Qin Wang, Xu Wang, Wei Ni, Shiping Chen, and Ren Ping Liu. 2026. Client-Cooperative Split Learning. *IEEE Transactions on Services Computing* (2026).

[22] Jieren Deng, Yijue Wang, Ji Li, Chenghong Wang, Chao Shang, Hang Liu, Sanguthevar Rajasekaran, and Caiwen Ding. 2021. Tag: Gradient attack on transformer-based language models. In *Findings of the association for computational linguistics: EMNLP 2021*. Association for Computational Linguistics, Punta Cana, Dominican Republic, 3600–3610.

[23] Minxin Du, Xiang Yue, Sherman SM Chow, Tianhao Wang, Chenyu Huang, and Huan Sun. 2023. Dp-forward: Fine-tuning and inference on language models with differential privacy in forward pass. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*. New York, NY, USA, 2665–2679.

[24] Qiang Duan, Shijing Hu, Ruijun Deng, and Zhihui Lu. 2022. Combined federated and split learning in edge computing for ubiquitous intelligence in internet of things: State-of-the-art and future directions. *Sensors* 22, 16 (2022), 5983.

[25] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating noise to sensitivity in private data analysis. In *Theory of cryptography conference*. Springer, Springer, Berlin, Heidelberg, 265–284.

[26] Ege Erdogan, Alptekin Küpçü, and A Ercument Cicek. 2022. Splitguard: Detecting and mitigating training-hijacking attacks in split learning. In *Proceedings of the 21st Workshop on Privacy in the Electronic Society*. Association for Computing Machinery, New York, NY, USA, 125–137.

[27] Ege Erdoğan, Alptekin Küpçü, and A Ercüment Çiçek. 2022. Unsplit: Data-oblivious model inversion, model stealing, and label inference attacks against split learning. In *Proceedings of the 21st Workshop on Privacy in the Electronic Society*. Association for Computing Machinery, New York, NY, USA, 115–124.

[28] Ege Erdoğan, Unat Tekşen, M Salih Çeliktenyıldız, Alptekin Küpçü, and A Ercüment Çiçek. 2024. SplitOut: Out-of-the-box training-hijacking detection in split learning via outlier detection. In *International Conference on Cryptology and Network Security*. Springer, Singapore, 118–142.

[29] Wenhao Fan, Penghui Chen, Xiongfei Chun, and Yuan'an Liu. 2025. MADRL-based model partitioning, aggregation control, and resource allocation for cloud-edge-device collaborative split federated learning. *IEEE Transactions on Mobile Computing* (2025).

[30] Xingyu Feng, Renqi Jia, Chengwen Luo, Victor CM Leung, and Weitao Xu. 2024. SLwF: A split learning without forgetting framework for Internet of Things. *IEEE Internet of Things Journal* 12, 9 (2024), 12008–12020.

[31] Chong Fu, Xuhong Zhang, Shouling Ji, Jinyin Chen, Jingzheng Wu, Shanqing Guo, Jun Zhou, Alex X Liu, and Ting Wang. 2022. Label inference attacks against vertical federated learning. In *31st USENIX security symposium (USENIX Security 22)*. 1397–1414.

[32] Fangcheng Fu, Xuanyu Wang, Jiawei Jiang, Huanran Xue, and Bui Cui. 2024. ProjPert: Projection-based perturbation for label protection in split learning based vertical federated learning. *IEEE Transactions on Knowledge and Data Engineering* 36, 7 (2024), 3417–3428.

[33] Jiayun Fu, Xiaojing Ma, Bin B Zhu, Pingyi Hu, Ruixin Zhao, Yaru Jia, Peng Xu, Hai Jin, and Dongmei Zhang. 2023. Focusing on Pinocchio's Nose: A Gradients Scrutinizer to Thwart Split-Learning Hijacking Attacks Using Intrinsic Attributes.. In *30th Annual Network and Distributed System Security Symposium*. San Diego, California, USA.

[34] Saurabh Gajbhiye, Priyanka Singh, and Shaifu Gupta. 2022. Data poisoning attack by label flipping on splitfed learning. In *International Conference on Recent Trends in Image Processing and Pattern Recognition*. Springer, Springer Nature Switzerland, Cham, 391–405.

[35] Siyu Gao. 2025. Communication-Efficient Split Federated Learning with Dynamic Feature Compression. In *2025 IEEE 6th International Seminar on Artificial Intelligence, Networking and Information Technology (AINIT)*. IEEE, 01–07.

[36] Xinben Gao and Lan Zhang. 2023. {PCAT}: Functionality and data stealing from split learning by {Pseudo-Client} attack. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 5271–5288.

[37] Yunqi Gao, Bing Hu, Mahdi Boloursaz Mashhadi, Wei Wang, and Mehdi Bennis. 2025. PipeSFL: A fine-grained parallelization framework for split federated learning on heterogeneous clients. *IEEE transactions on mobile computing* 24, 3 (2025), 1774–1791.

[38] Grzegorz Gawron and Philip Stubbings. 2022. *Feature space hijacking attacks against differentially private split learning.* arXiv:2201.04018 https://arxiv.org/abs/2201.04018

[39] Tianyu Gu, Kang Liu, Brendan Dolan-Gavitt, and Siddharth Garg. 2019. BadNets: Evaluating Backdooring Attacks on Deep Neural Networks. *IEEE Access* 7 (2019), 47230–47244.

[40] Yujie Gu, Richeng Jin, Xiaoyu Ji, Yier Jin, and Wenyuan Xu. 2026. *Differentially Private and Communication Efficient Large Language Model Split Inference via Stochastic Quantization and Soft Prompt.* arXiv:2602.11513 https://arxiv.org/abs/2602.11513

[41] Zixuan Gu, Qiufeng Fan, Long Sun, Yang Liu, and Xiaojun Ye. 2025. VFLAIR-LLM: A Comprehensive Framework and Benchmark for Split Learning of LLMs. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2 (KDD '25)*. Association for Computing Machinery, New York, NY, USA, 5470–5481.

[42] Otkrist Gupta and Ramesh Raskar. 2018. Distributed learning of deep neural network over multiple agents. *Journal of Network and Computer Applications* 116 (2018), 1–8.

[43] Dong-Jun Han, Hasnain Irshad Bhatti, Jungmoon Lee, and Jaekyun Moon. 2021. Accelerating federated learning with split learning on locally generated losses. In *ICML 2021 workshop on federated learning for user privacy and data confidentiality. ICML Board*.

[44] Dong-Jun Han, Do-Yeon Kim, Minseok Choi, Christopher G Brinton, and Jaekyun Moon. 2023. SplitGP: Achieving both generalization and personalization in federated learning. In *IEEE Conference on Computer Communications*. IEEE, 1–10.

[45] Dong-Jun Han, Do-Yeon Kim, Minseok Choi, David Nickel, Jaekyun Moon, Mung Chiang, and Christopher G Brinton. 2023. Federated split learning with joint personalization-generalization for inference-stage optimization in wireless edge networks. *IEEE Transactions on Mobile Computing* 23, 6 (2023), 7048–7065.

[46] Qiang He, Kaibin Wang, Zeqian Dong, Liang Yuan, Feifei Chen, Hai Jin, and Yun Yang. 2025. Hourglass: Enabling Efficient Split Federated Learning with Data Parallelism. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 1317–1333.

[47] Ying He, Zhili Shen, Jingyu Hua, Qixuan Dong, Jiacheng Niu, Wei Tong, Xu Huang, Chen Li, and Sheng Zhong. 2023. Backdoor attack against split neural network-based vertical federated learning. *IEEE Transactions on Information Forensics and Security* 19 (2023), 748–763.

[48] Zecheng He, Tianwei Zhang, and Ruby B Lee. 2019. Model inversion attacks against collaborative inference. In *Proceedings of the 35th annual computer security applications conference (ACSAC '19)*. ACM, Association for Computing Machinery, New York, NY, USA, 148–162.

[49] Xiaoyang Hou, Jian Liu, Jingyu Li, Yuhan Li, Jiawen Zhang, Wen-jie Lu, Cheng Hong, and Kui Ren. 2026. Ciphergpt: Secure two-party gpt inference. *IEEE Transactions on Dependable and Secure Computing* (2026), 1–16.

[50] Cheng-Yen Hsieh, Yu-Chuan Chuang, and An-Yeu Wu. 2022. C3-SL: Circular convolution-based batch-wise compression for communication-efficient split learning. In *IEEE 32nd International Workshop on Machine Learning for Signal Processing*. IEEE, 1–6.

[51] Chenghao Hu and Baochun Li. 2024. Menos: Split Fine-Tuning Large Language Models with Efficient GPU Memory Sharing. In *Proceedings of the 25th International Middleware Conference*. Association for Computing Machinery, New York, NY, USA, 185–198.

[52] Zhanyi Hu, Tianchen Zhou, Bingzhe Wu, Cen Chen, and Yanhao Wang. 2025. A review and experimental evaluation on split learning. *Future Internet* 17, 2 (2025), 87.

[53] Zhanyi Hu, Tianchen Zhou, Bingzhe Wu, Cen Chen, and Yanhao Wang. 2025. SLPerf: A Research Library and Benchmark Framework for Split Learning. In *2025 IEEE 41st International Conference on Data Engineering Workshops (ICDEW)*. IEEE, 33–36.

[54] Chao Huang, Geng Tian, and Ming Tang. 2023. When minibatch sgd meets splitfed learning: Convergence analysis and performance evaluation. arXiv:2308.11953 https://arxiv.org/abs/2308.11953

[55] Yirui Huang, Jia-Li Yin, Zhou Tan, Qiuxiang Wang, and Ximeng Liu. 2026. SLDP-LoRA: A privacy-preserving split learning framework with low-rank adaptation. *IEEE Transactions on Network Science and Engineering* 13 (2026), 2111–2127.

[56] Geetabai S Hukkeri, RH Goudar, GM Dhananjaya, Vijayalaxmi N Rathod, and Shilpa Ankalaki. 2025. A Comprehensive Survey on Split-Fed Learning: Methods, Innovations, and Future Directions. *IEEE Access* 13 (2025), 46312–46333.

[57] Joohyung Jeon and Joongheon Kim. 2020. Privacy-sensitive parallel split learning. In *2020 International Conference on Information Networking (ICOIN)*. IEEE, 7–9.

[58] Yanna Jiang, Guangsheng Yu, Qin Wang, Xu Wang, Baihe Ma, Caijun Sun, Wei Ni, and Ren Ping Liu. 2025. Split Unlearning. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*. Association for Computing Machinery, New York, NY, USA, 948–962.

[59] Praveen Joshi, Chandra Thapa, Seyit Camtepe, Mohammed Hasanuzzamana, Ted Scully, and Haithem Afli. 2021. Splitfed learning without client-side synchronization: Analyzing client-side split network portion size to overall performance. arXiv:2109.09246 https://arxiv.org/abs/2109.09246

[60] Halil Ibrahim Kanpak, Aqsa Shabbir, Esra Genç, Alptekin Küpçü, and Sinem Sav. 2024. CURE: Privacy-Preserving Split Learning Done Right. arXiv:2407.08977 https://arxiv.org/abs/2407.08977

[61] Sanjay Kariyappa and Moinuddin K Qureshi. 2023. Exploit: Extracting private labels in split learning. In *2023 IEEE conference on secure and trustworthy machine learning (SaTML)*. IEEE, 165–175.

[62] Tanveer Khan and Antonis Michalas. 2026. Oops!... They Stole it Again: Attacks on Split Learning. In *Proceedings of the 18th ACM Workshop on Artificial Intelligence and Security*. Association for Computing Machinery, New York, NY, USA, 123–135.

[63] Mohammad Kohankhaki, Ahmad Ayad, Mahdi Barhoush, and Anke Schmeink. 2023. Detecting data poisoning in split learning using intraclass-distance inflated loss. In *2023 IEEE Globecom Workshops (GC Wkshps)*. IEEE, 2091–2096.

[64] Jingtao Li, Adnan Siraj Rakin, Xing Chen, Zhezhi He, Deliang Fan, and Chaitali Chakrabarti. 2022. Ressfl: A resistance transfer framework for defending model inversion attack in split federated learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10194–10202.

[65] Oscar Li, Jiankai Sun, Xin Yang, Weihao Gao, Hongyi Zhang, Junyuan Xie, Virginia Smith, and Chong Wang. 2022. Label Leakage and Protection in Two-party Split Learning. In *International Conference on Learning Representations*.

[66] Zuguang Li, Shaohua Wu, Liang Li, and Songge Zhang. 2025. Energy-efficient split learning for fine-tuning large language models in edge networks. *IEEE Networking Letters* 7, 3 (2025), 176–180.

[67] Zuguang Li, Wen Wu, Shaohua Wu, and Wei Wang. 2024. Adaptive split learning over energy-constrained wireless edge networks. In *IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*. IEEE, 1–6.

[68] Ziang Li, Mengda Yang, Yaxin Liu, Juan Wang, Hongxin Hu, Wenzhe Yi, and Xiaoyang Xu. 2023. GAN you see me? enhanced data reconstruction attacks against split inference. *Advances in neural information processing systems* 36 (2023), 54554–54566.

[69] Dandan Liang, Jianing Zhang, Evan Chen, Zhe Li, Rui Li, and Haibo Yang. 2025. Towards Straggler-Resilient Split Federated Learning: An Unbalanced Update Approach. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. Red Hook, NY, USA.

[70] Xingzhu Liang, Yachen Xu, Yu-e Lin, and Chunjiong Zhang. 2025. Federated split learning via dynamic aggregation and homomorphic encryption on non-IID data. *The Journal of Supercomputing* 81, 1 (2025), 63.

[71] Yipeng Liang, Qimei Chen, Rongpeng Li, Guangxu Zhu, Muhammad Kaleem Awan, and Hao Jiang. 2026. Communication-and-Computation Efficient Split Federated Learning in Wireless Networks: Gradient Aggregation and Resource Management. *IEEE Transactions on Wireless Communications* 25 (2026), 1981–1995.

[72] Yunming Liao, Yang Xu, Hongli Xu, Lun Wang, Zhiwei Yao, and Chunming Qiao. 2024. MergeSFL: Split federated learning with feature merging and batch size regulation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2054–2067.

[73] Yunming Liao, Yang Xu, Hongli Xu, Zhiwei Yao, Liusheng Huang, and Chunming Qiao. 2024. Parallelsfl: A novel split federated learning framework tackling heterogeneity issues. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. New York, NY, USA, 845–860.

[74] Yunming Liao, Yang Xu, Hongli Xu, Zhiwei Yao, Lun Wang, and Chunming Qiao. 2023. Accelerating federated learning with data and model parallelism in edge computing. *IEEE/ACM Transactions on Networking* 32, 1 (2023), 904–918.

[75] Zheng Lin, Zhe Chen, Xianhao Chen, Wei Ni, and Yue Gao. 2026. HASFL: Heterogeneity-aware split federated learning over edge computing systems. *IEEE Transactions on Mobile Computing* (2026).

[76] Zheng Lin, Xuanjie Hu, Yuxin Zhang, Zhe Chen, Zihan Fang, Xianhao Chen, Ang Li, Praneeth Vepakomma, and Yue Gao. 2024. SplitLoRA: A Split Parameter-Efficient Fine-Tuning Framework for Large Language Models. arXiv:2407.00952 https://arxiv.org/abs/2407.00952

[77] Zehang Lin, Zhe Lin, Miao Yang, Jianhao Huang, Yuxin Zhang, Zihan Fang, Xia Du, Zhe Chen, Shunzhi Zhu, and Wei Ni. 2026. SL-ACC: A Communication-Efficient Split Learning Framework with Adaptive Channel-wise Compression. *IEEE Transactions on Vehicular Technology* (2026).

[78] Zheng Lin, Guanqiao Qu, Xianhao Chen, and Kaibin Huang. 2024. Split learning in 6G edge networks. *IEEE Wireless Communications* 31, 4 (2024), 170–176.

[79] Zheng Lin, Guanqiao Qu, Wei Wei, Xianhao Chen, and Kin K Leung. 2025. Adaptsfl: Adaptive split federated learning in resource-constrained edge networks. *IEEE Transactions on Networking* (2025).

[80] Zheng Lin, Wei Wei, Zhe Chen, Chan-Tong Lam, Xianhao Chen, Yue Gao, and Jun Luo. 2025. Hierarchical split federated learning: Convergence analysis and system optimization. *IEEE Transactions on Mobile Computing* (2025).

[81] Zheng Lin, Yuxin Zhang, Zhe Chen, Zihan Fang, Xianhao Chen, Praneeth Vepakomma, Wei Ni, Jun Luo, and Yue Gao. 2025. HSplitLoRA: A Heterogeneous Split Parameter-Efficient Fine-Tuning Framework for Large Language Models. arXiv:2505.02795 https://arxiv.org/abs/2505.02795

[82] Zheng Lin, Yuxin Zhang, Zhe Chen, Zihan Fang, Yanni Yang, Guoming Zhang, Huan Yang, Cong Wu, Xianhao Chen, and Yue Gao. 2025. Esl-leo: An efficient split learning framework over leo satellite networks. In *International Conference on Wireless Artificial Intelligent Computing Systems and Applications*. Springer, 344–357.

[83] Zheng Lin, Guangyu Zhu, Yiqin Deng, Xianhao Chen, Yue Gao, Kaibin Huang, and Yuguang Fang. 2024. Efficient parallel split learning over resource-constrained wireless edge networks. *IEEE Transactions on Mobile Computing* 23, 10 (2024), 9224–9239.

[84] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2025. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2025). arXiv:2412.19437 https://arxiv.org/abs/2412.19437

[85] Chenyu Liu, Zhaoyang Zhang, Zirui Chen, and Zhaohui Yang. 2025. Communication-Computation Pipeline Parallel Split Learning over Wireless Edge Networks. arXiv:2511.23167 https://arxiv.org/abs/2511.23167

[86] Junlin Liu and Xinchen Lyu. 2023. Distance-based online label inference attacks against split learning. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 1–5.

[87] Junlin Liu, Xinchen Lyu, Qimei Cui, and Xiaofeng Tao. 2024. Similarity-based label inference attack against training and inference of split learning. *IEEE Transactions on Information Forensics and Security* 19 (2024), 2881–2895.

[88] Junlin Liu, Xinchen Lyu, Longfei Zheng, Chenshan Ren, and Qimei Cui. 2026. PPoison: A Pluggable Poisoning attack against distributed training of split learning. *Future Generation Computer Systems* 175 (2026), 108045.

[89] Qi Liu, Zhilu Wang, Xiaokang Zhou, Yonghong Zhang, Xiaodong Liu, and Haiyang Lin. 2025. GSFL: A Privacy-Preserving Grouping-Split Federated Learning Approach in Resource-Constrained Edge Computing Scenarios. *ACM Transactions on Autonomous and Adaptive Systems* 20 (2025), 1 – 29.

[90] Tiantian Liu, Hongwei Yao, Tong Wu, Zhan Qin, Feng Lin, Kui Ren, and Chun Chen. 2024. Mitigating privacy risks in LLM embeddings from embedding inversion. arXiv:2511.23167 https://arxiv.org/html/2411.05034v1

[91] Zihan Liu, Yizhen Wang, Rui Wang, and Sai Wu. 2025. DualGuard: A Parameter Space Transformation Approach for Bidirectional Defense in Split-Based LLM Fine-Tuning. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria, 17065–17080.

[92] Wen-jie Lu, Zhicong Huang, Zhen Gu, Jingyu Li, Jian Liu, Cheng Hong, Kui Ren, Tao Wei, and WenGuang Chen. 2023. Bumblebee: Secure two-party inference framework for large transformers. *Cryptology ePrint Archive* (2023).

[93] Linjun Luo and Xinglin Zhang. 2024. Federated split learning via mutual knowledge distillation. *IEEE Transactions on Network Science and Engineering* 11, 3 (2024), 2729–2741.

[94] Sida Luo, Fangchao Yu, Lina Wang, Bo Zeng, Zhi Pang, and Kai Zhao. 2023. Feature sniffer: A stealthy inference attacks framework on split learning. In *International conference on artificial neural networks*. Springer, Cham, 66–77.

[95] Song Lyu, Zheng Lin, Guanqiao Qu, Xianhao Chen, Xiaoxia Huang, and Pan Li. 2023. Optimal resource allocation for u-shaped parallel split learning. In *2023 IEEE Globecom Workshops (GC Wkshps)*. IEEE, 197–202.

[96] Jian Ma, Xinchen Lyu, Jun Jiang, Qimei Cui, Haipeng Yao, and Xiaofeng Tao. 2025. SplitFrozen: Split learning with device-side model frozen for fine-tuning LLM on heterogeneous resource-constrained devices. *IEEE Communications Magazine* (2025).

[97] Peihua Mai, Ran Yan, Zhe Huang, Youjia Yang, and Yan Pang. 2024. Split-and-denoise: protect large language model inference with local differential privacy. In *Proceedings of the 41st International Conference on Machine Learning*. 34281–34302.

[98] Yunlong Mao, Zexi Xin, Zhenyu Li, Jue Hong, Qingyou Yang, and Sheng Zhong. 2023. Secure split learning against property inference, data reconstruction, and feature space hijacking attacks. In *European Symposium on Research in Computer Security*. Springer, 23–43.

[99] Matea Marinova, Marija Poposka, Zoran Hadzi-Velkov, and Valentin Rakovic. 2025. Optimal Cut Layer Bounds for Split Learning. *IEEE Communications Letters* (2025).

[100] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. 2025. Large Language Models: A Survey. arXiv:2402.06196 https://arxiv.org/abs/2402.06196

[101] Yujia Mu and Cong Shen. 2023. Communication and storage efficient federated split learning. In *ICC 2023-IEEE International Conference on Communications*. IEEE, 2976–2981.

[102] Srijith Nair, Michael Lin, Peizhong Ju, Amirreza Talebi, Elizabeth Serena Bentley, and Jia Liu. 2025. FSL-SAGE: Accelerating Federated Split Learning via Smashed Activation Gradient Estimation. In *Forty-second International Conference on Machine Learning* (Vancouver, Canada).

[103] Seungeun Oh, Hyelin Nam, Jihong Park, Praneeth Vepakomma, Ramesh Raskar, Mehdi Bennis, and Seong-Lyun Kim. 2023. Mix2SFL: Two-way mixup for scalable, accurate, and communication-efficient split federated learning. *IEEE Transactions on Big Data* 10, 3 (2023), 238–248.

[104] Seungeun Oh, Jihong Park, Praneeth Vepakomma, Sihun Baek, Ramesh Raskar, Mehdi Bennis, and Seong-Lyun Kim. 2022. Locfedmix-sl: Localize, federate, and mix for improved scalability, convergence, and latency in split learning. In *Proceedings of the ACM Web Conference 2022* (Virtual Event, Lyon, France). 3347–3357.

[105] Yongjeong Oh, Jaeho Lee, Christopher G Brinton, and Yo-Seb Jeon. 2025. Communication-efficient split learning via adaptive feature-wise compression. *IEEE Transactions on Neural Networks and Learning Systems* (2025).

[106] Shraman Pal, Mansi Uniyal, Jihong Park, Praneeth Vepakomma, Ramesh Raskar, Mehdi Bennis, Moongu Jeon, and Jinho Choi. 2021. Server-Side Local Gradient Averaging and Learning Rate Acceleration for Scalable Split Learning. arXiv:2112.05929 https://arxiv.org/abs/2112.05929

[107] Xudong Pan, Mi Zhang, Shouling Ji, and Min Yang. 2020. Privacy risks of general-purpose language models. In *2020 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1314–1331.

[108] Dario Pasquini, Giuseppe Ateniese, and Massimo Bernaschi. 2021. Unleashing the tiger: Inference attacks on split learning. In *Proceedings of the 2021 ACM SIGSAC conference on computer and communications security*. New York, NY, USA, 2113–2129.

[109] George-Liviu Pereteanu, Amir Alansary, and Jonathan Passerat-Palmbach. 2022. Split HE: Fast Secure Inference Combining Split Learning and Homomorphic Encryption. arXiv:2202.13351 https://arxiv.org/abs/2202.13351

[110] Ngoc Duy Pham, Alsharif Abuadbba, Yansong Gao, Khoa Tran Phan, and Naveen Chilamkurti. 2023. Binarizing split learning for data privacy enhancement and computation reduction. *IEEE Transactions on Information Forensics and Security* 18 (2023), 3088–3100.

[111] Ngoc Duy Pham, Khoa T Phan, and Naveen Chilamkurti. 2024. Enhancing accuracy-privacy trade-off in differentially private split learning. *IEEE Transactions on Emerging Topics in Computational Intelligence* 9, 1 (2024), 988–1000.

[112] Ngoc Duy Pham, Tran Khoa Phan, Alsharif Abuadbba, Yansong Gao, Van-Doan Nguyen, and Naveen Chilamkurti. 2025. Split learning without local weight sharing to enhance client-side data privacy. *IEEE Transactions on Dependable and Secure Computing* (2025).

[113] Yuwen Pu, Zhuoyuan Ding, Jiahao Chen, Chunyi Zhou, Qingming Li, Chunqiang Hu, and Shouling Ji. 2024. Dullahan: Stealthy Backdoor Attack against Without-Label-Sharing Split Learning. arXiv:2405.12751 https://arxiv.org/abs/2405.12751

[114] Xinchi Qiu, Ilias Leontiadis, Luca Melis, Alex Sablayrolles, and Pierre Stock. 2024. Evaluating Privacy Leakage in Split Learning. arXiv:2305.12997 https://arxiv.org/abs/2305.12997

[115] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. 2018. Improving language understanding by generative pre-training. (2018).

[116] Boris Radovič, Mohammed Aljahdali, Marco Canini, Veljko Pejović, and Zuhair Khayyat. 2024. Train your cake and eat it too! Repurposing collaborative training to tailor LLMs to private data without sharing. In *Workshop on Efficient Systems for Foundation Models II@ ICML2024*.

[117] Boris Radovič, Marco Canini, Samuel Horváth, Veljko Pejović, and Praneeth Vepakomma. 2025. Towards a Unified Framework for Split Learning. In *Proceedings of the 5th Workshop on Machine Learning and Systems*. 183–191.

[118] Phillip Rieger, Alessandro Pegoraro, Kavita Kumari, Tigist Abera, Jonathan Knauer, and Ahmad-Reza Sadeghi. 2025. SafeSplit: A Novel Defense Against Client-Side Backdoor Attacks in Split Learning. In *Proceedings of the Network and Distributed System Security Symposium*. Reston, VA.

[119] Eric Samikwa, Antonio Di Maio, and Torsten Braun. 2022. Ares: Adaptive resource-aware split learning for internet of things. *Computer Networks* 218 (2022), 109380.

[120] Aqsa Shabbir, Halil İbrahim Kanpak, Alptekin Küpçü, and Sinem Sav. 2025. A Taxonomy of Attacks and Defenses in Split Learning. arXiv:2505.05872 https://arxiv.org/abs/2505.05872

[121] Jiawei Shao and Jun Zhang. 2020. Bottlenet++: An end-to-end approach for feature compression in device-edge co-inference systems. In *2020 IEEE International Conference on Communications Workshops (ICC Workshops)*. IEEE, 1–6.

[122] Jinglong Shen, Nan Cheng, Xiucheng Wang, Feng Lyu, Wenchao Xu, Zhi Liu, Khalid Aldubaikhy, and Xuemin Shen. 2023. Ringsfl: An adaptive split federated learning towards taming client heterogeneity. *IEEE Transactions on Mobile Computing* 23, 5 (2023), 5462–5478.

[123] Xicong Shen, Yang Liu, Yi Liu, Peiran Wang, Huiqi Liu, Jue Hong, Bing Duan, Zirui Huang, Yunlong Mao, Ye Wu, et al. 2025. SAP: Privacy-Preserving Fine-Tuning on Language Models with Split-and-Privatize Framework. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*. 502–510.

[124] Haonan Shi, Tu Ouyang, and An Wang. 2025. Navigating the Designs of Privacy-Preserving Fine-tuning for Large Language Models. In *Companion Proceedings of the ACM on Web Conference 2025*. New York, NY, USA, 1298–1302.

[125] Shaohuai Shi, Qing Yang, Yang Xiang, Shuhan Qi, and Xuan Wang. 2022. An Efficient Split Fine-tuning Framework for Edge and Cloud Collaborative Learning. arXiv:2211.16703 https://arxiv.org/abs/2211.16703

[126] Yunmeng Shu, Pengwenlong Gu, Cédric Adjih, Chung Shue Chen, and Ahmed Serhrouchni. 2024. Dynsplit: A dynamic split learning scheme for 5g-enpowered metaverse. In *2024 IEEE International Conference on Metaverse Computing, Networking, and Applications (MetaCom)*. IEEE, 214–221.

[127] Veronika Stephanie, Ibrahim Khalil, and Mohammed Atiquzzaman. 2023. Digital twin enabled asynchronous SplitFed learning in E-healthcare systems. *IEEE Journal on Selected Areas in Communications* 41, 11 (2023), 3650–3661.

[128] Yunrui Sun, Gang Hu, Yinglei Teng, and Dunbo Cai. 2025. Split federated learning over heterogeneous edge devices: Algorithm and optimization. In *2025 IEEE Wireless Communications and Networking Conference (WCNC)*. IEEE, 01–06.

[129] Behrad Tajalli, Oğuzhan Ersoy, and Stjepan Picek. 2023. On feasibility of server-side backdoor attacks on split learning. In *2023 IEEE security and privacy workshops (SPW)*. IEEE, 84–93.

[130] Vo Phuc Tinh, Tran Anh Khoa, Pham Duc Lam, Nguyen Hoang Nam, Duc Ngoc Minh Dang, Duy-Dong Le, Thai-Thinh Dang, Van-Luong Nguyen, Thanh-Qui Pham, and Thai-Binh Nguyen. 2025. SMixSL: The Smashed-Mixture Technique for Split Learning With Localizable Features. *IEEE Transactions on Emerging Topics in Computational Intelligence* (2025).

[131] Joana Tirana, Dimitra Tsigkari, George Iosifidis, and Dimitris Chatzopoulos. 2024. Workflow optimization for parallel split learning. In *IEEE INFOCOM 2024-IEEE Conference on Computer Communications*. IEEE, 1331–1340.

[132] Meng Tong, Kejiang Chen, Jie Zhang, Yuang Qi, Weiming Zhang, Nenghai Yu, Tianwei Zhang, and Zhikun Zhang. 2025. Inferdpt: Privacy-preserving inference for black-box large language models. *IEEE Transactions on Dependable and Secure Computing* (2025).

[133] Le Hoang Trung, Tan Y Nguyen, Duy Dong Le, Thai Thinh Dang, and Tran Anh Khoa. 2025. Latency-Aware Split Learning Optimization via Genetic Algorithms. In *Proceedings of the 6th Workshop on Intelligent Cross-Data Analysis and Retrieval*. 14–19.

[134] Praneeth Vepakomma, Otkrist Gupta, Tristan Swedish, and Ramesh Raskar. 2018. Split learning for health: Distributed deep learning without sharing raw patient data. arXiv:1812.00564 https://arxiv.org/abs/1812.00564

[135] Praneeth Vepakomma, Abhishek Singh, Otkrist Gupta, and Ramesh Raskar. 2020. NoPeek: Information leakage reduction to share activations in distributed deep learning. In *2020 International Conference on Data Mining Workshops (ICDMW)*. IEEE, 933–942.

[136] Manas Wadhwa, Gagan Raj Gupta, Ashutosh Sahu, Rahul Saini, and Vidhi Mittal. 2023. Pfsl: Personalized & fair split learning with data & label privacy for thin clients. In *2023 IEEE/ACM 23rd International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*. IEEE, 377–390.

[137] Wei Wan, Yuxuan Ning, Shengshan Hu, Lulu Xue, Minghui Li, Leo Yu Zhang, and Hai Jin. 2024. Misa: Unveiling the vulnerabilities in split federated learning. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 6435–6439.

[138] Jianyu Wang, Hang Qi, Ankit Singh Rawat, Sashank Reddi, Sagar Waghmare, Felix X. Yu, and Gauri Joshi. 2022. FedLite: A Scalable Approach for Federated Learning on Resource-constrained Clients. arXiv:2201.11865 https://arxiv.org/abs/2201.11865

[139] Teng Wang, Lindong Zhai, Tengfei Yang, Zhucheng Luo, and Shuanggen Liu. 2024. Selective privacy-preserving framework for large language models fine-tuning. *Information Sciences* 678 (2024), 121000.

[140] Tianhao Wang, Yuheng Zhang, and Ruoxi Jia. 2021. Improving robustness to model inversion attacks via mutual information regularization. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 35. 11666–11673.

[141] Haoqi Wu, Wei Dai, Wang Li, and Qiang Yan. 2025. Cape: Context-Aware Prompt Perturbation Mechanism with Differential Privacy. In *International Conference on Machine Learning*. PMLR, 67184–67201.

[142] Jiasheng Wu, Jingjing Zhang, Zheng Lin, Zhe Chen, Xiong Wang, Wenjun Zhu, and Yue Gao. 2025. SFL-LEO: Asynchronous Split-Federated Learning Design for LEO Satellite-Ground Network Framework. arXiv:2504.13479 https://arxiv.org/abs/2504.13479

[143] Maoqiang Wu, Guoliang Cheng, Dongdong Ye, Jiawen Kang, Rong Yu, Yuan Wu, and Miao Pan. 2023. Federated split learning with data and label privacy preservation in vehicular networks. *IEEE Transactions on Vehicular Technology* 73, 1 (2023), 1223–1238.

[144] Wen Wu, Mushu Li, Kaige Qu, Conghao Zhou, Xuemin Shen, Weihua Zhuang, Xu Li, and Weisen Shi. 2023. Split learning over wireless networks: Parallel design and resource management. *IEEE Journal on Selected Areas in Communications* 41, 4 (2023), 1051–1066.

[145] Xiaodong Wu, Henry Yuan, Xiangman Li, Jianbing Ni, and Rongxing Lu. 2024. Evaluating security and robustness for split federated learning against poisoning attacks. *IEEE Transactions on Information Forensics and Security* (2024).

[146] Peng Xi, Shaoliang Peng, and Wenjuan Tang. 2025. IBAS: Imperceptible Backdoor Attacks in Split Learning with Limited Information. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 27715–27722.

[147] Jiaqi Xia, Meng Wu, and Pengyong Li. 2025. SFML: A personalized, efficient, and privacy-preserving collaborative traffic classification architecture based on split learning and mutual learning. *Future Generation Computer Systems* 162 (2025), 107487.

[148] Zeke Xia, Ming Hu, Dengke Yan, Ruixuan Liu, Anran Li, Xiaofei Xie, and Mingsong Chen. 2025. MultiSFL: Towards Accurate Split Federated Learning via Multi-Model Aggregation and Knowledge Replay. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 914–922.

[149] Shangyu Xie, Xin Yang, Yuanshun Yao, Tianyi Liu, Taiqing Wang, and Jiankai Sun. 2023. Label Inference Attack against Split Learning under Regression Setting. arXiv:2301.07284 https://arxiv.org/abs/2301.07284

[150] Ce Xu, Jinxuan Li, Yuan Liu, Yushi Ling, and Miaowen Wen. 2023. Accelerating split federated learning over wireless communication networks. *IEEE Transactions on Wireless Communications* 23, 6 (2023), 5587–5599.

[151] Guangwei Xu, Jianbo Lu, Xinru Wang, Yang Lu, Mei Cao, and Mengying Zhao. 2024. CSFL: Enhancing Splitfed Learning with Clustering on Non-IID Data. In *2024 IEEE International Conference on High Performance Computing and Communications (HPCC)*. IEEE, 791–798.

[152] Xiaoyang Xu, Mengda Yang, Wenzhe Yi, Ziang Li, Juan Wang, Hongxin Hu, Yong Zhuang, and Yaxin Liu. 2024. A stealthy wrongdoer: Feature-oriented reconstruction attack against split learning. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. Seattle, WA, USA, 12130–12139.

[153] Dengke Yan, Ming Hu, Zeke Xia, Yanxin Yang, Jun Xia, Xiaofei Xie, and Mingsong Chen. 2024. Have Your Cake and Eat It Too: Toward Efficient and Accurate Split Federated Learning. arXiv:2311.13163 https://arxiv.org/abs/2311.13163

[154] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. arXiv:2505.09388 https://arxiv.org/abs/2505.09388

[155] Jiarong Yang and Yuan Liu. 2025. GAS: Generative Activation-Aided Asynchronous Split Federated Learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 21956–21964.

[156] Jiarong Yang and Yuan Liu. 2025. SCALA: Split Federated Learning with Concatenated Activations and Logit Adjustments. arXiv:2405.04875 https://arxiv.org/abs/2405.04875

[157] Xin Yang, Jiankai Sun, Yuanshun Yao, Junyuan Xie, and Chong Wang. 2022. Differentially Private Label Protection in Split Learning. arXiv:2203.02073 https://arxiv.org/abs/2203.02073

[158] Zhiwei Yao, Ji Qi, Yang Xu, Yunming Liao, Hongli Xu, and Lun Wang. 2025. PairingFL: Efficient Federated Learning With Model Splitting and Client Pairing. *IEEE Transactions on Networking* (2025).

[159] Wenzhe Yi, Mengda Yang, Juan Wang, Hongxin Hu, Ziang Li, Xiaoyang Xu, Yu He, and Yao Wang. 2025. Membership inference attacks against split inference via knowledge transfer. *Neurocomputing* (2025), 132247.

[160] Fangchao Yu, Lina Wang, Bo Zeng, Kai Zhao, Zhi Pang, and Tian Wu. 2023. How to backdoor split learning. *Neural Networks* 168 (2023), 326–336.

[161] Fangchao Yu, Lina Wang, Bo Zeng, Kai Zhao, Tian Wu, and Zhi Pang. 2024. SIA: A sustainable inference attack framework in split learning. *Neural Networks* 171 (2024), 396–409.

[162] Fangchao Yu, Bo Zeng, Kai Zhao, Zhi Pang, and Lina Wang. 2024. Chronic poisoning: Backdoor attack against split learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 16531–16538.

[163] Tianjun Yuan, Jiaxiang Geng, Pengchao Han, Xianhao Chen, and Bing Luo. 2025. Flexible Personalized Split Federated Learning for On-Device Fine-Tuning of Foundation Models. arXiv:2508.10349 https://arxiv.org/abs/2508.10349

[164] Xiang Yue, Minxin Du, Tianhao Wang, Yaliang Li, Huan Sun, and Sherman SM Chow. 2021. Differential privacy for text analytics via natural text sanitization. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. 3853–3866.

[165] Aysha-Thahsin Zahir-Ismail and Raj Mani Shukla. 2025. Analyzing the vulnerabilities in Split Federated Learning: assessing the robustness against data poisoning attacks. *Scientific Reports* 15 (2025).

[166] Bo Zeng, Sida Luo, Fangchao Yu, Geying Yang, Kai Zhao, and Lina Wang. 2025. GAN-based data reconstruction attacks in split learning. *Neural Networks* 185 (2025), 107150.

[167] Lan Zhang, Xinben Gao, Yaliang Li, and Yunhao Liu. 2024. Functionality and data stealing by pseudo-client attack and target defenses in split learning. *IEEE Transactions on Dependable and Secure Computing* 22, 1 (2024), 84–100.

[168] Mingjin Zhang, Jiannong Cao, Yuvraj Sahni, Xiangchun Chen, and Shan Jiang. 2024. Resource-efficient Parallel Split Learning in Heterogeneous Edge Computing. In *2024 International Conference on Computing, Networking and Communications (ICNC)*. IEEE, 794–798.

[169] Songge Zhang, Guoliang Cheng, Zuguang Li, and Wen Wu. 2024. Splitllm: Hierarchical split learning for large language model over wireless network. In *2024 IEEE Globecom Workshops (GC Wkshps)*. IEEE, 1–6.

[170] Songge Zhang, Guoliang Cheng, Wen Wu, Xinyu Huang, Lingyang Song, and Xuemin Shen. 2025. Split fine-tuning for large language models in wireless networks. *IEEE Journal of Selected Topics in Signal Processing* (2025).

[171] Songge Zhang, Haoyu Tu, Zuguang Li, Shengbo Liu, Shaofeng Li, Wen Wu, and Xuemin Sherman Shen. 2023. Cluster-hsfl: a cluster-based hybrid split and federated learning. In *2023 IEEE/CIC International Conference on Communications in China (ICCC)*. IEEE, 1–2.

[172] Zihan Zhang, Philip Rodgers, Peter Kilpatrick, Ivor Spence, and Blesson Varghese. 2024. PiPar: Pipeline parallelism for collaborative machine learning. *J. Parallel and Distrib. Comput.* 193 (2024), 104947.

[173] Zihan Zhang, Leon Wong, and Blesson Varghese. 2025. Ampere: Communication-Efficient and High-Accuracy Split Federated Learning. arXiv:2507.07130 https://arxiv.org/abs/2507.07130

[174] Bo Zhao, Konda Reddy Mopuri, and Hakan Bilen. 2020. idlg: Improved deep leakage from gradients. arXiv:2301.07284 http://arxiv.org/abs/2001.02610

[175] Kai Zhao, Xiaowei Chuo, Fangchao Yu, Bo Zeng, Zhi Pang, and Lina Wang. 2024. SplitAUM: Auxiliary model-based label inference attack against split learning. *IEEE Transactions on Network and Service Management* (2024).

[176] Kai Zhao, Chen Zhu, Mingzhe Chen, Chongwen Huang, Zhaohui Yang, and Zhaoyang Zhang. 2025. SflLLM: Efficient Split Federated Learning for Large Language Model over Wireless Networks. In *GLOBECOM 2025-2025 IEEE Global Communications Conference*. IEEE, 1835–1840.

[177] Fei Zheng, Chaochao Chen, Lingjuan Lyu, and Binhui Yao. 2023. Reducing communication for split learning by randomized top-k sparsification. In *Proceedings of the Thirty-Second International Joint Conference on Artificial Intelligence*. 4665–4673.

[178] Wenxuan Zhou, Zhihao Qu, Shen-Huan Lyu, Miao Cai, and Baoliu Ye. 2024. Mask-encoded sparsification: Overcoming biased gradients for communication-efficient split learning. In *Proceedings of the 27th European Conference on Artificial Intelligence*. 2806–2813.

[179] Xin Zhou, Yi Lu, Ruotian Ma, Tao Gui, Yuran Wang, Yong Ding, Yibo Zhang, Qi Zhang, and Xuan-Jing Huang. 2023. TextObfuscator: Making pre-trained language model a privacy protector via obfuscating word representations. In *Findings of the association for computational linguistics: ACL 2023*. Toronto, Canada, 5459–5473.

[180] Guogang Zhu, Xuefeng Liu, Jianwei Niu, Shaojie Tang, Xinghao Wu, and Jiayuan Zhang. 2024. Dualfed: enjoying both generalization and personalization in federated learning via hierachical representations. In *Proceedings of the 32nd ACM International Conference on Multimedia*. New York, NY, USA, 11060–11069.

[181] Ligeng Zhu, Zhijian Liu, and Song Han. 2019. Deep leakage from gradients. *Advances in neural information processing systems* 32 (2019).

[182] Xiaochen Zhu, Xinjian Luo, Yuncheng Wu, Yangfan Jiang, Xiaokui Xiao, and Beng Chin Ooi. 2025. Passive Inference Attacks on Split Learning via Adversarial Regularization. In *Network and Distributed System Security (NDSS) Symposium 2025*. San Diego, CA.

[183] Tianyuan Zou, Yang Liu, and Ya-Qin Zhang. 2023. Mutual Information Regularization for Vertical Federated Learning. arXiv:2301.01142 https://arxiv.org/abs/2301.01142